

Legge dei grandi numeri e teorema del limite centrale (somme)

Sia X una variabile aleatoria distribuita secondo una densita' di probabilita' $f(x)$ qualunque

$$\mu = \int_{-\infty}^{+\infty} xf(x)dx \quad \left(\sum x_i f_i \right) \quad \text{Caso discreto}$$

$$\sigma^2 = \int_{-\infty}^{+\infty} (x - \mu)^2 f(x)dx \quad \left(\sum (x_i - \mu)^2 f_i \right) < +\infty$$

Caso continuo

Allora se costruiamo la variabile

$$Y_N = x_1 + x_2 + \dots + x_N$$

Si ha che (se la varianza σ^2 e' finita) :

- ☐ $Y_N \rightarrow N\mu$ per $N \rightarrow +\infty$
- ☐ $\sigma_{Y_N}^2 \rightarrow N\sigma^2$ per $N \rightarrow +\infty$

Il teorema del limite centrale inoltre afferma che la Y_N tende ad una distribuzione gaussiana

$$f(Y_N) \rightarrow \frac{1}{\sqrt{2\pi N\sigma^2}} e^{-\frac{(Y_N - N\mu)^2}{2N\sigma^2}} \quad \text{per } N \rightarrow +\infty$$

Verifica del teorema del limite centrale



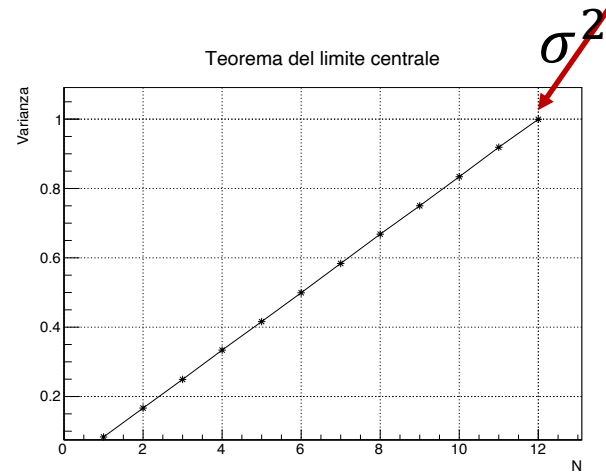
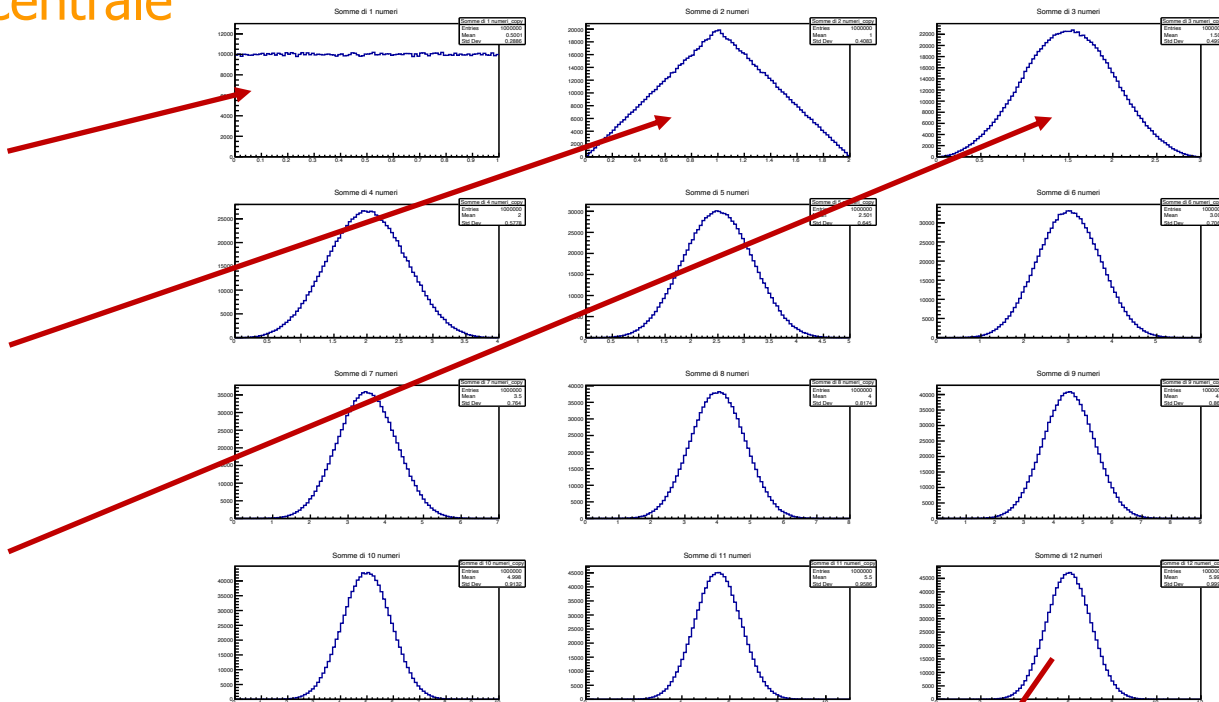
somme



somme



somme



Legge dei grandi numeri e teorema del limite centrale (medie)

Sia X una variabile aleatoria distribuita secondo una densita' di probabilita' $f(x)$ qualunque

$$\mu = \int_{-\infty}^{+\infty} xf(x)dx \quad \left(\sum x_i f_i \right) \quad \text{Caso discreto}$$

$$\sigma^2 = \int_{-\infty}^{+\infty} (x - \mu)^2 f(x)dx \quad \left(\sum (x_i - \mu)^2 f_i \right) < +\infty$$

Caso continuo

Allora se costruiamo la variabile

$$Y_N = \frac{x_1 + x_2 + \dots + x_N}{N}$$

Si ha che (se la varianza σ^2 e' finita) :

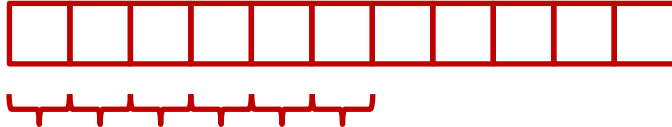
- ☐ $Y_N \rightarrow \mu$ per $N \rightarrow +\infty$
- ☐ $\sigma_{Y_N}^2 \rightarrow \frac{\sigma^2}{N}$ per $N \rightarrow +\infty$

Il teorema del limite centrale inoltre afferma che la Y_N tende ad una distribuzione gaussiana

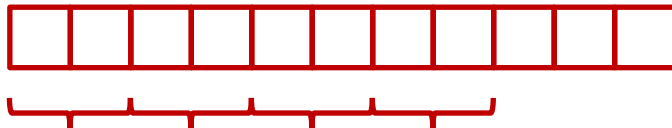
$$f(Y_N) \rightarrow \frac{1}{\sqrt{2\pi \left(\frac{\sigma^2}{N} \right)}} e^{-\frac{(Y_N - \mu)^2}{2 \left(\frac{\sigma^2}{N} \right)}} \quad \text{per } N \rightarrow +\infty$$

Montecarlo

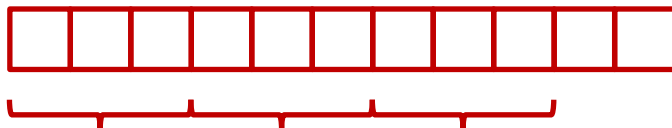
Verifica del teorema del limite centrale



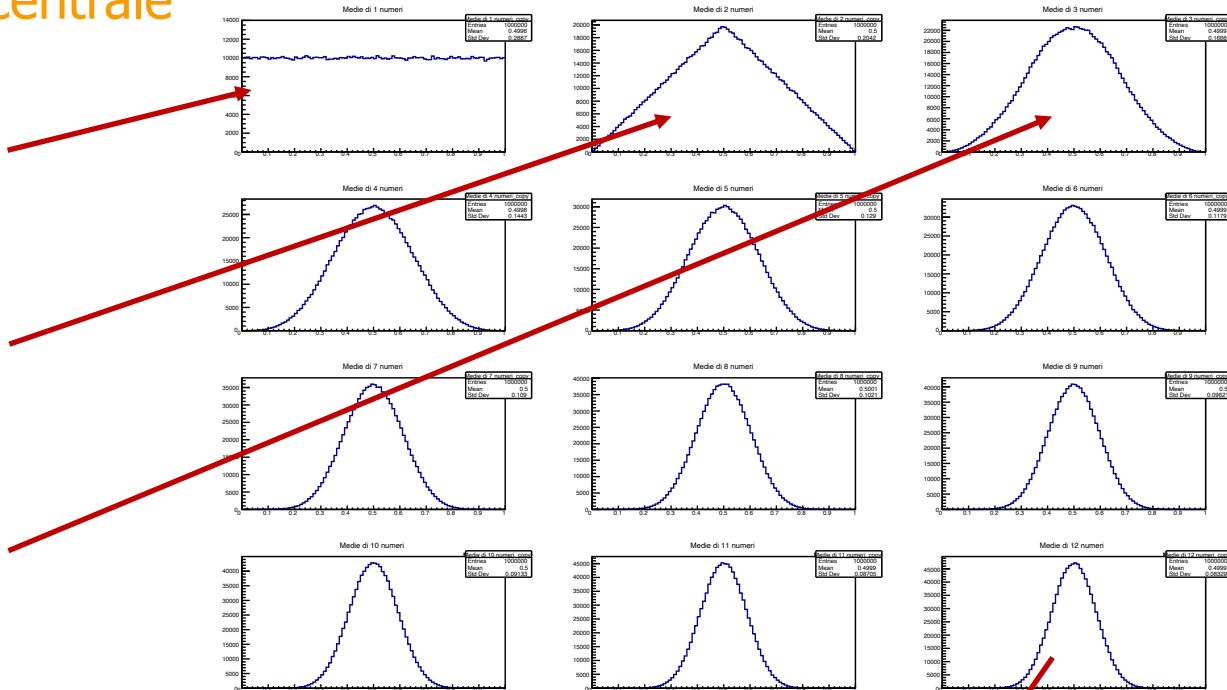
media



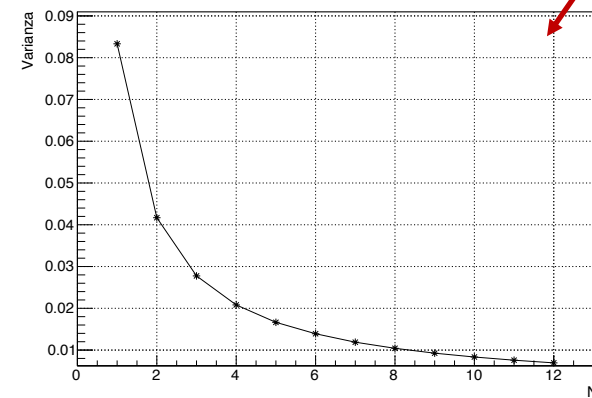
media



media



Teorema del limite centrale



Generatori di numeri casuali in STD

Nella nostra implementazione ogni numero viene generato da una chiamata ad un certo metodo (non generiamo direttamente una sequenza)

2^{31} (attenzione non $2E31$!)

```
class RandomGen {
public:
    RandomGen(unsigned int seed) {
        m_a = 1664525;
        m_c = 1013904223;
        m_m = 1<<31;
        m_seed = seed;
        m_seed_original = seed ;
    }

    unsigned int getSeedOriginal() { return m_seed_original ;} ;

    double Rand( ) {
        m_seed = (m_a*m_seed+m_c)%m_m;
        return double(m_seed)/double(m_m);
    }

    double Uniform( double min, double max ) {
        return min + (max-min)* Rand();
    }

    double GaussAR(double mean, double sigma) {
        double xmin= mean-5*sigma ;
        double xmax= mean+5*sigma;
        double fmax = 1/sqrt( 2*M_PI*sigma* sigma) ;
        double x=0, y=0;

        do {
            x = Uniform( xmin , xmax ) ;
            y = Uniform( 0 , fmax ) ;
        } while ( y > 1/sqrt( 2*M_PI*sigma* sigma) * exp( - pow(x-mean,2)/(2*sigma*sigma) ) );

        return x ;
    }

private:
    unsigned int m_a, m_c, m_m;
    unsigned int m_seed , m_seed_original;
};
```

Possiamo conservare il seed originale

Non `double(m_seed/m_m)` !

$r_{i+1} = (a * r_i + c) \% (m)$

Attenzione all'AR :
dobbiamo continuare
ad estrarre finché non
otteniamo un numero
x (!!) buono da
restituire

Per la gaussiana g possiamo
assumere $[-5\sigma; +5\sigma]$ come
range di estrazione e calcolare fmax
come $g(\text{mean})$

Si potrebbe pensare ad un generic AcceptReject che
accetti in input una FunzioneBase&, fmax, xmin e xmax

Generatori di numeri casuali e metodi Montecarlo (parte 2)

Laboratorio Trattamento Numerico dei Dati Sperimentali

L. Carminati
Universita' degli studi di Milano

Generatori di numeri casuali in STD

```
// ...
#include <random>

// add all ROOT #include

int main() {

    std::mt19937 e1 (2.) ;

    std::uniform_real_distribution<> uniform_dist(1, 6);
    TH1F hunif("uniform","Uniform distribution",100, 0, 10);
    for ( int k = 0 ; k < 10000 ; k++ ) hunif.Fill( uniform_dist(e1) );

    std::normal_distribution<> normal_dist(2, 1);
    TH1F hgauss("gauss","Normal distribution",100, -10, 10);
    for (int n = 0; n < 10000; n++) hgauss.Fill( normal_dist( e1 ) );

    std::exponential_distribution<> expo_dist(0.5);
    TH1F hexpo("expo","Exponential distribution",100, 0, 20);
    for (int n = 0; n < 10000; n++) hexpo.Fill( expo_dist( e1 ) );

    std::poisson_distribution<> pois_dist(4);
    TH1F hpois("Poisson","Poisson distribution",15, 0, 15);
    for (int n = 0; n < 10000; n++) hpois.Fill( pois_dist( e1 ) );

    // show distributions

    TCanvas * can = new TCanvas("distributions","Random numbers generator");
    can->Divide(2,2);
    can->cd(1);
    hunif.Draw();

    // plot all the others

    return 0;
}
```

Marsenne-Twister generator

Shape the flat generation on the desired output distribution

Generatori di numeri casuali in ROOT

```
#include "TRandom3.h"

// add all ROOT #include

int main() {

    TRandom3 rand(2.);

    TH1F Rhunif("Uniform(ROOT)","Uniform(ROOT)",100, 0, 10);
    for (int n = 0; n < 10000; n++) Rhunif.Fill( rand.Uniform(1,6) );
    TH1F Rhgaus("gaus(ROOT)","Normal distribution(ROOT)",100, -10, 10);
    for (int n = 0; n < 10000; n++) Rhgaus.Fill( rand.Gaus(2,1) );
    TH1F Rhexpo("expo(ROOT)","Exponential distribution(ROOT)",100, 0, 20);
    for (int n = 0; n < 10000; n++) Rhexpo.Fill( rand.Exp(0.5) );
    TH1F Rhpois("Poisson(ROOT)","Poisson distribution(ROOT)",15, 0, 15);
    for (int n = 0; n < 10000; n++) Rhpois.Fill( rand.Poisson(4) );

    // show distributions

    TCanvas * Rcan = new TCanvas("distributions(ROOT)","Random numbers generator(ROOT)");
    Rcan->Divide(2,2);
    Rcan->cd(1);
    Rhunif.Draw();

    // plot all the others

    return 0;
}
```

Costruisci un generatore

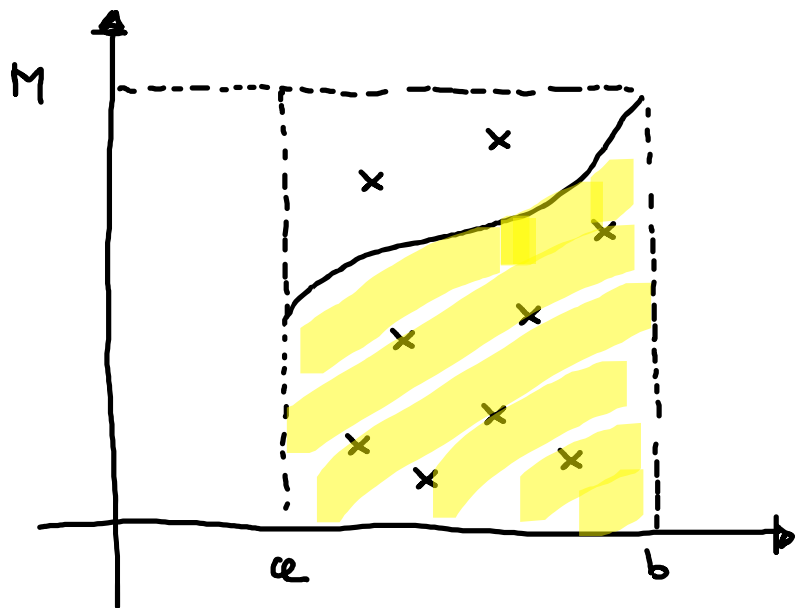
Usare diversi metodi per le diverse distribuzioni desiderate

Usare sempre
librerie
esistenti !

(eccetto che per l'esame di TNDS)

Integrazione Montecarlo (I): metodo Hit or Miss

Una interessante applicazione dei numeri casuali: tecniche Montecarlo per l'integrazione numerica



- ❑ Estraggo un numero casuale $\bar{x}$ uniformemente distribuito tra a e b
- ❑ Estraggo un numero casuale $\bar{y}$ uniformemente distribuito tra 0 e M (M = massimo della f(x))
- ❑ Ad ogni estrazione di una coppia $(\bar{x}, \bar{y})$ incremento un contatore N_{TOT}
- ❑ Se $\bar{y} < f(\bar{x})$ incremento un secondo contatore N_{HIT}

Intuitivamente uno stimatore dell'integrale puo' essere scritto

$$\hat{I} = (b - a)M \frac{N_{HIT}}{N_{TOT}}$$

Attenzione nell'implementazione :
rapporto tra interi

In pratica peso l'area del rettangolo $M(b-a)$ con la frazione di punti che stanno sotto la funzione

Integrazione Montecarlo (I) : metodo Hit or Miss

La garanzia che $\hat{I}$ sia un buono stimatore del valore dell'integrale (ie : converge all'integrale vero per numero di estrazioni crescente) ci viene dal teorema del limite centrale

- ❑ Il processo di estrazione puo' essere rappresentato da una variabile aleatoria x_i che mappi lo spazio degli eventi elementari in $\mathbb{R}$

$$x_i = \begin{cases} \text{successo} & \rightarrow +1 \\ \text{insuccesso} & \rightarrow 0 \end{cases}$$

- ❑ Assegnamo una probabilita' alla variabile aleatoria che abbiamo appena costruito (ad essere precisi la probabilita' andrebbe assegnata agli eventi di $\mathbb{F}$)

$$x_i = \begin{cases} +1 \text{ (successo)} & \rightarrow p(+1) = \frac{A_{int}}{A_{tot}} (= p) \\ 0 \text{ (insuccesso)} & \rightarrow p(0) = 1 - p(+1) = 1 - p \end{cases}$$

- ❑ Calcoliamo il valore di aspettazione e la varianza della variabile aleatoria x_i

$$\mu = \sum x_i p_i = 1p + 0(1 - p) = p \left(= \frac{A_{int}}{A_{tot}} \right)$$

$$\sigma^2 = \langle (x - \mu)^2 \rangle = (1 - p)^2 p + (0 - p)^2 (1 - p) = p(1 - p)$$

Integrazione Montecarlo (I) : metodo Hit or Miss

- ❑ Rappresentiamo N estrazioni con la variabile aleatoria

$$Y_N = \sum x_i (= N_{HIT})$$

- ❑ Per il teorema del limite centrale

$$Y_N \rightarrow N\mu = Np = N \frac{A_{int}}{A_{tot}}$$

$$\frac{Y_N}{N} A_{tot} = \frac{N_{HIT}}{N} A_{tot} = \hat{I} \rightarrow A_{int}$$

Il nostro stimatore dell'integrale converge all'integrale vero !

- ❑ Cosa succede all'errore ? Ancora per il teorema del limite centrale

$$\sigma_{Y_N}^2 = N\sigma_x^2 \Rightarrow \sigma_{Y_N} = \sqrt{N}\sigma_x$$

$$\sigma_I = \frac{\sigma_{Y_N}}{N} A_{tot} = \frac{\sqrt{N}\sigma_x}{N} A_{tot} = \frac{\sigma_x A_{tot}}{\sqrt{N}} = \frac{k}{\sqrt{N}}$$

Errore di un metodo MC scala come $k/\sqrt{N}$ con N numero di estrazioni che ho effettuato

Integrazione Montecarlo (II) : metodo della media

Vogliamo calcolare l'integrale di una funzione $f(x)$ in $[a,b]$

- ❑ Consideriamo una variabile aleatoria x distribuita secondo una densita' di probabilita' $p(x)$
- ❑ Estraiamo dei numeri casuali x_i con la distribuzione $p(x)$ e valutiamo $f(x_i)$.
- ❑ Costruiamo ora la variabile aleatoria Y_N come somma delle N variabili $f(x_i)$

$$Y_N = \frac{\sum f(x_i)}{N}$$

- ❑ Per il teorema del limite centrale

$$Y_N \rightarrow \langle f(x) \rangle = \int_{-\infty}^{+\infty} f(x)p(x)dx$$

Law of unconscious statistician (*)

- ❑ Se ora scelgo la $p(x)$ come distribuzione uniforme tra $[a,b]$

$$p(x) = \begin{cases} \frac{1}{b-a} & x \in [a,b] \\ 0 & x \notin [a,b] \end{cases} \quad Y_N \rightarrow \langle f(x) \rangle = \int_{-\infty}^{+\infty} f(x)p(x)dx = \int_a^b f(x) \frac{1}{b-a} dx$$

$$\hat{I}_N = \frac{\sum f(x_i)}{N} (b-a) \rightarrow \int_a^b f(x)dx \quad N \rightarrow +\infty$$

Integrazione Montecarlo (II) : metodo della media

❑ L'algoritmo in sintesi :

- ❑ Estraiamo N numeri casuali x_i uniformemente distribuiti $[a, b]$
- ❑ Calcoliamo I valori della funzione in questi punti $f(x_i)$
- ❑ Calcoliamo lo stimatore $\hat{I}_N = \frac{\sum f(x_i)}{N} (b - a)$ e questo tende proprio all'integrale vero per $N \rightarrow +\infty$

❑ Come si comporta l'errore :

Teorema del limite centrale

$$\sigma_{I_N}^2 \rightarrow \frac{\sigma_f^2}{N} (b - a)^2$$

$$\sigma_I = \frac{\sigma_f (b - a)}{\sqrt{N}} \quad \text{con} \quad \sigma_f = \sqrt{\frac{1}{N-1} \sum (f(x_i) - \bar{f})^2}$$

Ancora scaling con $1/\sqrt{N}$

Stimatore della deviazione standard : possiamo stimare l'errore da un singolo calcolo di integrale (senza repliche) !!

(*) Law of unconscious statistician

- ❑ Vogliamo mostrare che il valore di aspettazione di una funzione g di variabile aleatoria x distribuita con una pdf pari a $f(x)$ e' $\int_{-\infty}^{+\infty} g(x)f(x)dx$
- ❑ Incominciamo a considerare una variabile aleatoria Y non negativa. Mostriamo che vale

$$\langle Y \rangle = \int_0^{+\infty} P(Y \geq y) dy$$

- ❑ Infatti possiamo scrivere il membro di destra in questo modo :

$$\begin{aligned} \int_0^{+\infty} P(Y \geq y) dy &= \int_0^{+\infty} \int_y^{+\infty} f_Y(x) dx = \\ \int_0^{+\infty} \left(\int_0^x dy \right) f_Y(x) dx &= \int_0^{+\infty} x f_Y(x) dx = \langle Y \rangle \end{aligned}$$

(*) Law of unconscious statistician

- ❑ Consideriamo una regione di $g(x)$ in cui $g(x) \geq 0$: secondo quanto mostrato in precedenza

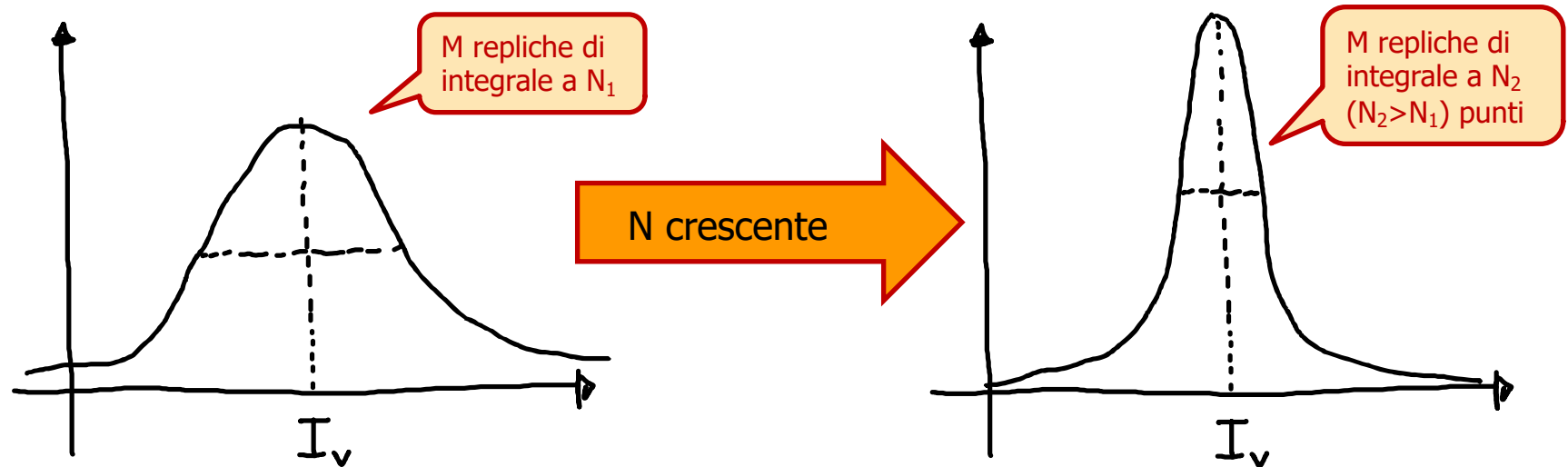
$$\begin{aligned}\langle g(x) \rangle &= \int_0^{+\infty} P(g(x) \geq y) dy \\ \int_0^{+\infty} dy \int_{x:g(x)>y} f(x) dx &= \int_{x:g(x)>y} \int_0^{g(x)} dy f(x) dx \\ &= \int_{x:g(x)>y} g(x) f(x) dx\end{aligned}$$

- ❑ Possiamo poi procedere per intervalli in cui la funzione non cambia di segno e concludere che

$$\langle g(x) \rangle = \int_{-\infty}^{+\infty} g(x) f(x) dx$$

Senso dell'errore di un metodo Montecarlo

- ❑ Se effettuo il calcolo di un integrale con il metodo Hit or Miss con un set di N punti casuali due volte non otterrò lo stesso valore ! (se il set di punti estratti e' diverso)
- ❑ Il senso dell'errore di un integrale con metodo MC e' il seguente : se ripeto il calcolo dell'integrale a N punti M volte e li metto in un istogramma (con M entries) i valori si distribuiranno secondo una gaussiana centrata attorno al valore vero dell'integrale e di larghezza σ_I



- ❑ Per stimare run-time l'errore non ha nessun senso fare $I_N - I_{2N}$: paradossalmente una stima dell'integrale a N punti potrebbe essere piu' vicina all'integrale vero si una stima a $2N$ punti !

Senso dell'errore di un metodo Montecarlo : calcolo a errore fissato !

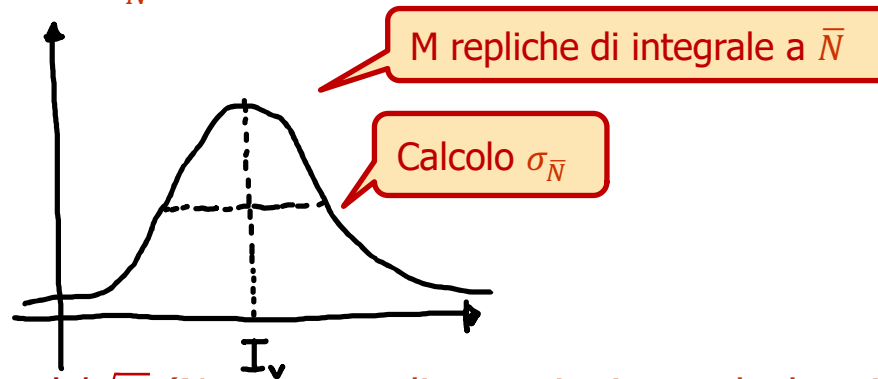
- ❑ Il senso dell'errore e' statistico : fissato il valore di N si devono eseguire M repliche dell'integrazione e guardare la deviazione standard dei valori ottenuti
- ❑ Un possibile algoritmo per calcolare l'integrale con precisione $\tilde{\sigma}$:
 - ❑ Ripetere M volte il calcolo dell'integrale con $\bar{N}$ punti e calcolare la deviazione standard $\sigma_{\bar{N}}$
 - ❑ Conoscendo l'andamento dell'errore stimare la costante $k = \sigma_{\bar{N}} \sqrt{\bar{N}}$
 - ❑ Stimare il numero di punti $\tilde{N}$ che sarebbero necessari per avere $\tilde{\sigma}$:

$$\tilde{N} = \frac{k^2}{\tilde{\sigma}^2} = \frac{\sigma_{\bar{N}}^2 \bar{N}}{\tilde{\sigma}^2}$$

Integrale a errore fissato :

□ Una possibile procedura per un generico metodo di integrazione MC

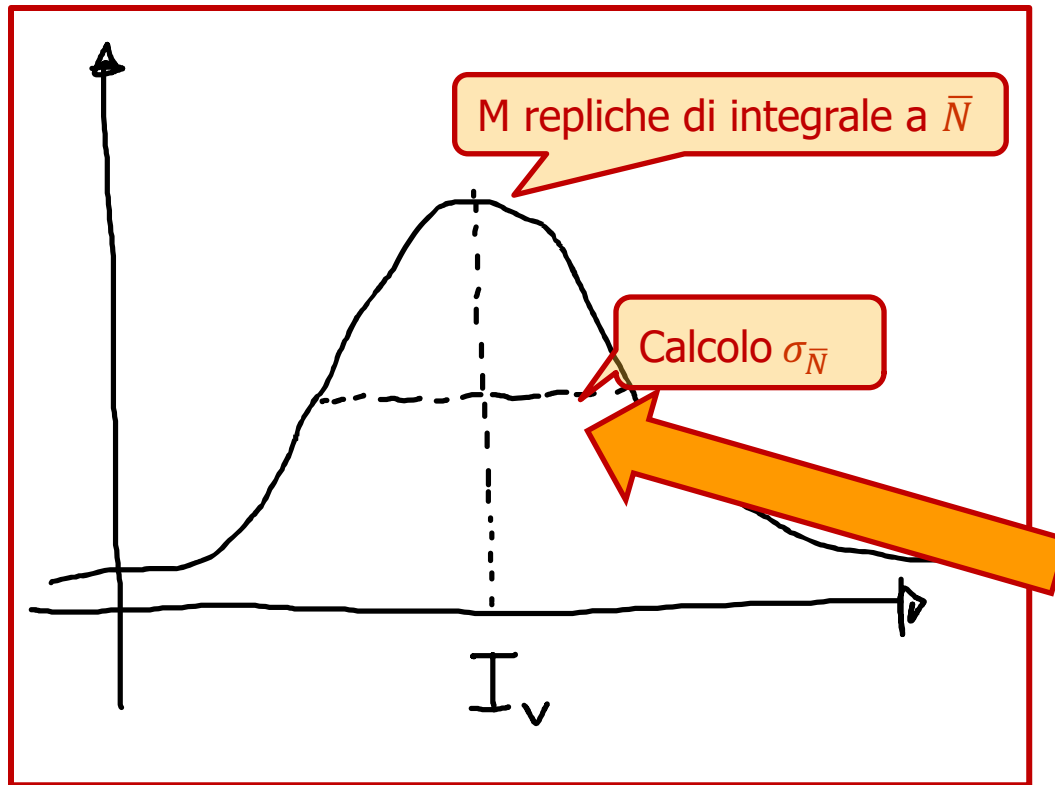
1. Scelgo un numero di estrazioni (arbitrario) $\bar{N}$ e calcolo M volte (diciamo ~ 10000) l'integrale (ciascun integrale viene dall'estrazione di $\bar{N}$ punti)
2. Dal vettore degli M integrali calcolo $\sigma_{\bar{N}}$



3. Sapendo che l'errore scala come $k/\sqrt{N}$ (N =numero di estrazioni pe calcolare il singolo integrale) uso la formuletta per determinare $\tilde{N}$ necessario per ottenere un errore di $\tilde{\sigma}$
4. Effettuo il calcolo di un singolo integrale con $\tilde{N}$ estrazioni. Il senso di questo integrale : e' un possibile valore estratto da una distribuzione con larghezza $\tilde{\sigma}$! (non dice quanto sia davvero vicino al valore vero)

□ NOTA importante : nel caso del metodo della media la valutazione preliminare di $\tilde{\sigma}$ in 1. e 2. non e' necessaria, basta una sola valutazione dell'integrale ! $\sigma_I = \frac{\sigma_f(b-a)}{\sqrt{N}}$ con $\sigma_f = \sqrt{\frac{1}{N-1} \sum (f(x_i) - \bar{f})^2}$

Integrale a errore fissato : metodo della media



Per il metodo della media vale la procedura generale della slide precedente ma si può pensare ad un algoritmo molto più semplice: in questo caso infatti non è strettamente necessario fare repliche per stimare $\sigma_{\bar{N}}$. Basta calcolare da un singolo integrale a $\bar{N}$ punti

$$\sigma_I = \frac{\sigma_f(b-a)}{\sqrt{\bar{N}}} \quad \sigma_f = \sqrt{\frac{1}{\bar{N}-1} \sum (f(x_i) - \bar{f})^2}$$

e σ_I coincide con $\sigma_{\bar{N}}$ (entro gli errori statistici)

□ Nella funzione che calcola l'integrale mentre si calcola $\bar{f}$ predisporre anche il calcolo di σ_f

□ A questo punto si estrapola a $\tilde{N}$ necessari per ottenere $\tilde{\sigma}$ come nel caso della procedura generale

Calcolo integrale con metodi MC : opzione 1 (generatore come oggetto)

- ❑ Classe che prevede un generatore di numeri casuali come data membro
- ❑ Costruttore di `IntegraleMC` accetta un seed come input
- ❑ Come inizializzare il generatore di numeri casuali con il seed dal costruttore di `IntegraleMC` ?

```
class IntegraleMC {
public:
    IntegraleMC(unsigned int seed) : m_gen(seed)
    {
        m_errore = 0;
    }

    virtual double Integra(const FunzioneBase* f, double inf , double sup , unsigned int punti) = 0 ;

    double GetErrore() const {return m_errore;}
    unsigned int GetN() const {return m_punti;}

protected:
    RandomGen m_gen;
    double m_errore;
    unsigned int m_punti;
};
```

Nel costruttore di `MCIntegrator` (che accetta il seme come input) posso inserire una lista di inizializzazione che mi permette di specificare quale costruttore di `RandomGen` invocare

Generatore di numeri casuali come data membro (oggetto)

Calcolo integrale con metodi MC : opzione 2 (generatore come puntatore)

- ❑ Classe che prevede un generatore di numeri casuali come puntatore
- ❑ Costruttore di `IntegraleMC` accetta un seed come input
- ❑ Come inizializzare il generatore di numeri casuali con il seed dal costruttore di `IntegraleMC` ?

```
class IntegraleMC {  
public:  
    IntegraleMC(unsigned int seed)  
    {  
        m_gen = new RandomGen(seed);  
        m_errore = 0;  
    }  
  
    virtual double Integra(const FunzioneBase* f, double inf, double sup, unsigned int punti) = 0 ;  
  
    double GetErrore() const {return m_errore;}  
    unsigned int GetN() const {return m_punti;}  
  
protected:  
    RandomGen * m_gen;  
    double m_errore;  
    unsigned int m_punti;  
};
```

Nel costruttore di `MCIntegrator` (che accetta il seme `seed` come input) costruisco un generatore (`new RandomGen(seed)`) e lo aggancio a `m_gen`

Generatore di numeri casuali come data membro (puntatore)

- ❑ Implementiamo il metodo della media e hit-or-miss come classi derivate da IntegraleMC

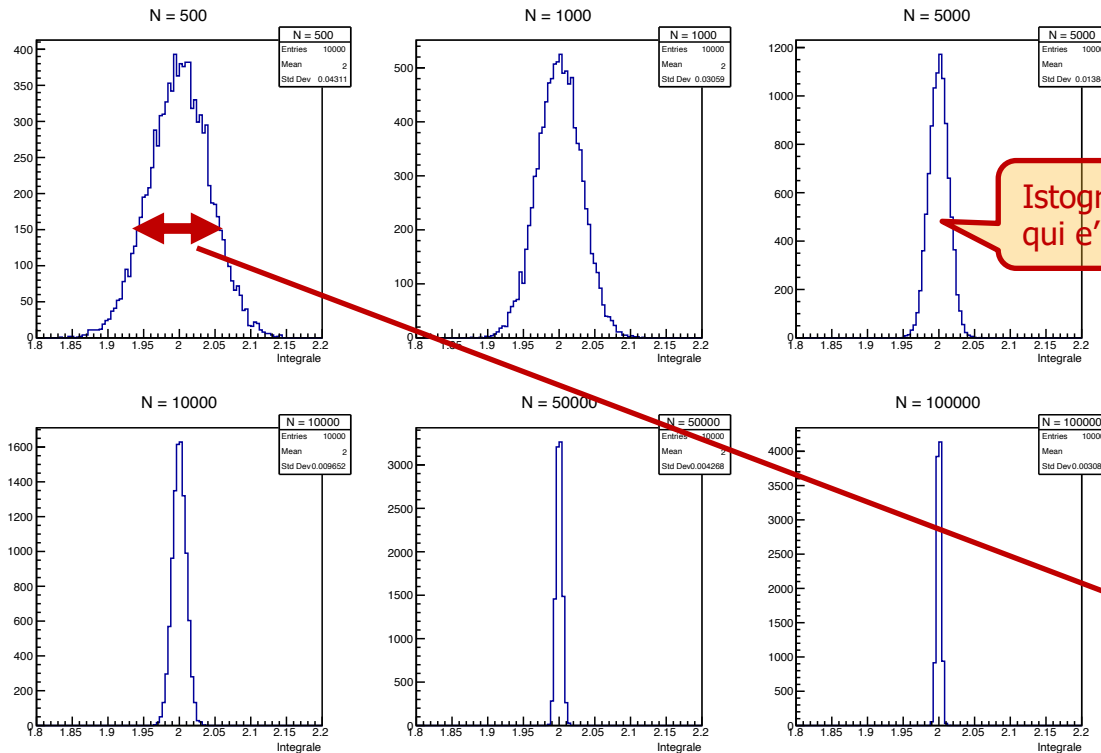
```
class IntegratoreMedia : public IntegraleMC {  
public:  
    IntegratoreMedia(unsigned int seed) : IntegraleMC(seed) { ; };  
    virtual double Integra ( const FunzioneBase& f, double inf , double sup , unsigned int punti, double fmax ) {  
        // ....  
    };  
};
```

Se vogliamo salvare questa struttura di classe madre astratta e classe figlia concreta dobbiamo avere tra i piedi $f_{\max}$ anche per il metodo della media (anche se strettamente non necessario)

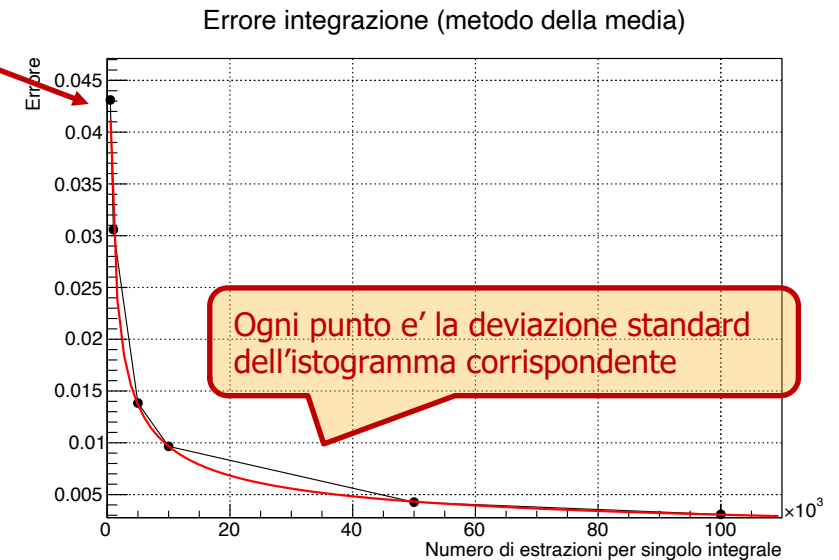
Verifica andamento errore metodi integrazione MC : esercizio10.2

1. Calcolare 10000 volte il valore dell'integrale di $\sin(x)$ tra $[0, \pi]$ utilizzando il metodo della media a 100 punti e fare un grafico (istogramma) della distribuzione dei valori ottenuti.
2. Estendere il punto precedente calcolando 10000 volte il valore dell'integrale di $\sin(x)$ tra $[0, \pi]$ utilizzando il metodo della media a N punti con N pari a 500, 1000, 5000, 10000, 50000 e 100000 punti. Per ogni valore di N produrre il grafico della distribuzione dei 10000 valori ottenuti. [NOTA: poichè il calcolo degli integrali con N molto elevato potrebbe richiedere un certo tempo, potrebbe essere utile salvare in diversi files i valori degli integrali calcolati per un determinato N e svolgere i punti successivi con un secondo programma che utilizzi come input i files di integrali del programma precedente.]
3. Stimare l'errore sul calcolo dell'integrale a 500, 1000, 5000, 10000, 50000 e 100000 punti come deviazione standard dei 10000 valori calcolati per ogni N . Far un grafico di questo errore in funzione di N .
4. Assumendo che l'andamento dell'errore sia noto (del tipo $k/\sqrt{N}$) si determini quanti punti sono necessari per ottenere una precisione di 0.001.
5. Si ripeta lo stesso lavoro con il metodo hit-or-miss.

Verifica andamento errore metodi integrazione MC : esercizio10.2



Istogramma con 10000 entries : ogni entry qui e' un integrale calcolato con 5000 punti.



Esercizio10.2 : implementazione in due steps

Questo e' un caso tipico in cui il codice e' costituito da una parte lenta (il calcolo ripetuto degli integrali) e una parte veloce (l'estrazione della deviazione standard dagli istogrammi e la creazione del grafico finale). In genere si tende a separare le due parti in codici diversi.

1. Codice che calcola gli integrali e li scrive su files : per esempio un file con i famosi 10000 numeri per ogni valore di N ($N=100, 1000, 5000, 10000, 50000, 100000$). Questa parte di codice potrebbe durare ordine minuti.
2. Codice che apre i files e produce i grafici finali :
 1. Potremmo volerlo modificare molte volte a seconda dell'aspetto che vogliamo dare ai grafici
 2. Potremmo volerlo implementare usando altri tools o linguaggi (c++ e ROOT, python e matplotlib, gnuplot ...)

Esercizio10.2 : calcolo integrali

Non provato !

```
int main() {  
    vector<int> cases { 500, 1000, 5000, 10000, 50000, 100000 } ;  
  
    IntegratoreMedia MCIntegrator ;  
    Seno mysin ;  
    fstream f ;  
  
    for ( auto i = 0 ; i < cases.size() ; i++ ) {  
        cout << "==> Running integration with N = " << cases[i] << " points" << endl ;  
  
        int nmax = 10000 ;  
  
        string nomefile = .... ;  
        f.open ( nomefile, ios::out ) ;  
  
        for ( int k = 0 ; k < nmax ; k++ ) {  
            f << MCIntegrator.Integra( mysin , /* ... */ , cases[i], /* ... */ ) << endl ;  
            if ( ! (k%1000 ) ) cout << "Working " << double(k)/nmax * 100 << "%" << endl ;  
        }  
        f.close() ;  
    }  
}
```

Casi che voglio esplorare :
numero di estrazioni per il
calcolo del singolo integrale

Costruisco il nome del file e
ci scrivo dentro gli integrali

Scrivo un file per ogni caso
che voglio esplorare

Esercizio10.2 : lettura e grafici

Non provato !

```
int main() {  
    TApplication app("app",0,0) ;  
    fstream f;  
    vector<int> cases { 100, 1000, 5000, 10000, 50000, 100000 } ;  
    vector<TH1F*> vhistos( cases.size() );  
    double varTemp = 0;  
    // ....  
    for ( auto i = 0 ; i < cases.size() ; i++ ) {  
        string nomefile = "integrali_"+to_string( cases[i] )+".dat";  
        string nomehisto = "histo"+to_string(cases[i]);  
        TH1F* h = new ( nomehisto.c_str(), nomehisto.c_str(),100,1.8,2.2 );  
        h->StatOverflows( kTRUE );  
        f.open ( nomefile, ios::in );  
        // read from file and fill histogram  
        f.close() ;  
        // append and draw  
        vhistos[i] = h ;  
        vhistos[i]->GetXaxis()->SetTitle("Integrale");  
        vhistos[i]->Draw();  
    }  
}
```

Vector di TH1F*

Casi che voglio esplorare :
numero di estrazioni per il
calcolo del singolo integrale

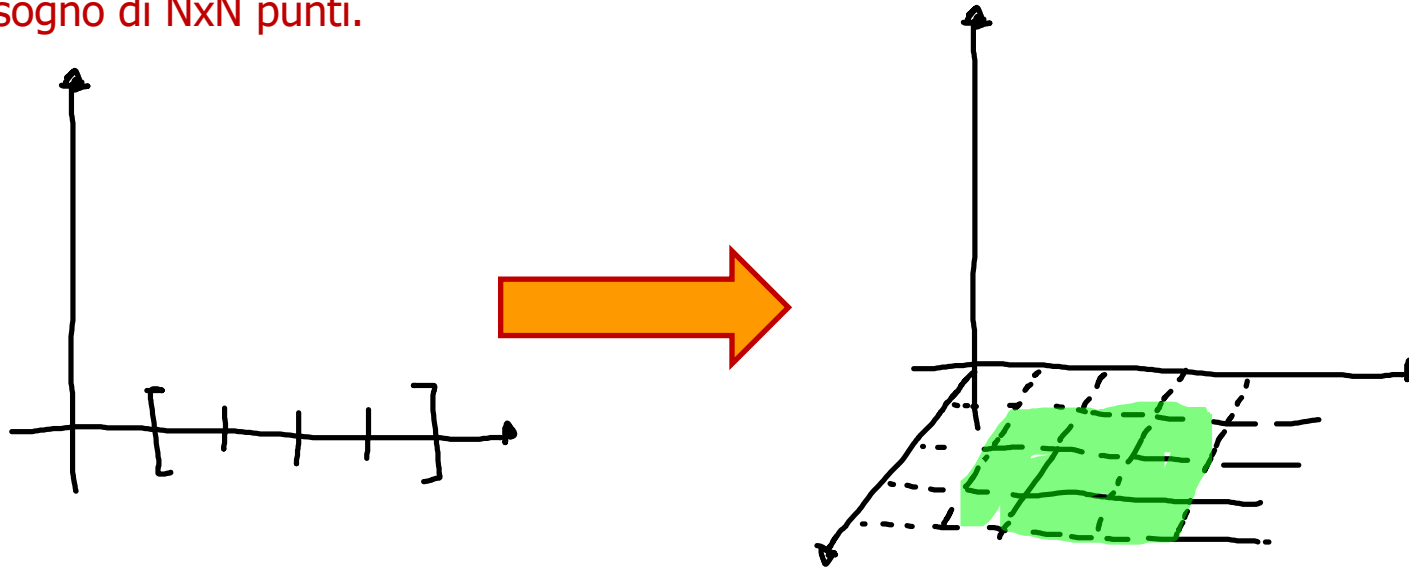
Costruisco puntatori
a istogrammi

Memorizzo I puntatori
a istogrammi

```
RFLAGS:='root-config --cflags' `root-config --libs`  
  
all: read write  
  
read: read.cpp ...  
      g++ -o read ... ${RFLAGS}  
  
write: write.cpp ...  
      g++ -o write ... ${RFLAGS}  
  
clean :  
      rm *.o
```

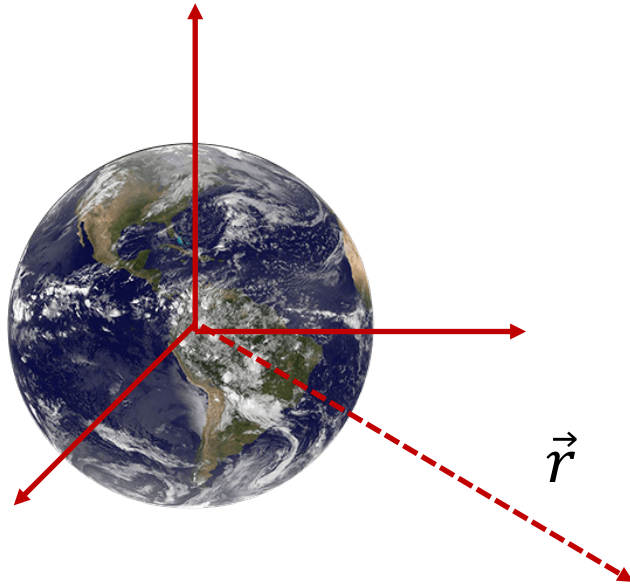
Integrali multidimensionali

- ❑ Applichiamo il metodo dei trapezoidi ad un integrale monodimensionale : ci aspettiamo che l'errore scali come $1/N^2$ (potete riprendere le trasparenze della lezione dedicate)
- ❑ Cosa succede se calcoliamo un integrale con il medesimo metodo ? Per avere la stessa precisione avremo bisogno di $N \times N$ punti.



- ❑ Per N fissato la precisione del metodo e' ridotta in funzione del numero di dimensioni : $\frac{k_1}{N^{\frac{d}{2}}}$ (trapezodi mentre va come $\frac{k_2}{N^{\frac{d}{4}}}$ nel caso del metodo di Simpson)
- ❑ Nei metodi MC errore $\sim \frac{k}{\sqrt{N}}$ indipendentemente dal numero di dimensioni del problema !

Interessa un integrale 3D ?



$$V(\vec{r}) = - \int dx' dy' dz' G \frac{\rho(\vec{r}')}{|\vec{r} - \vec{r}'|}$$



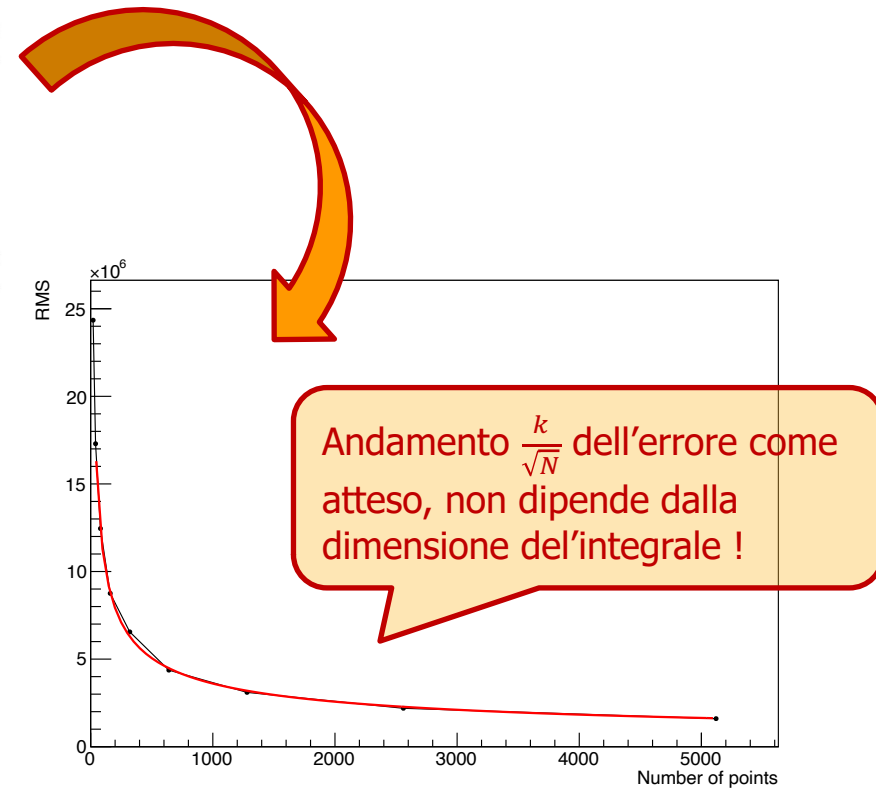
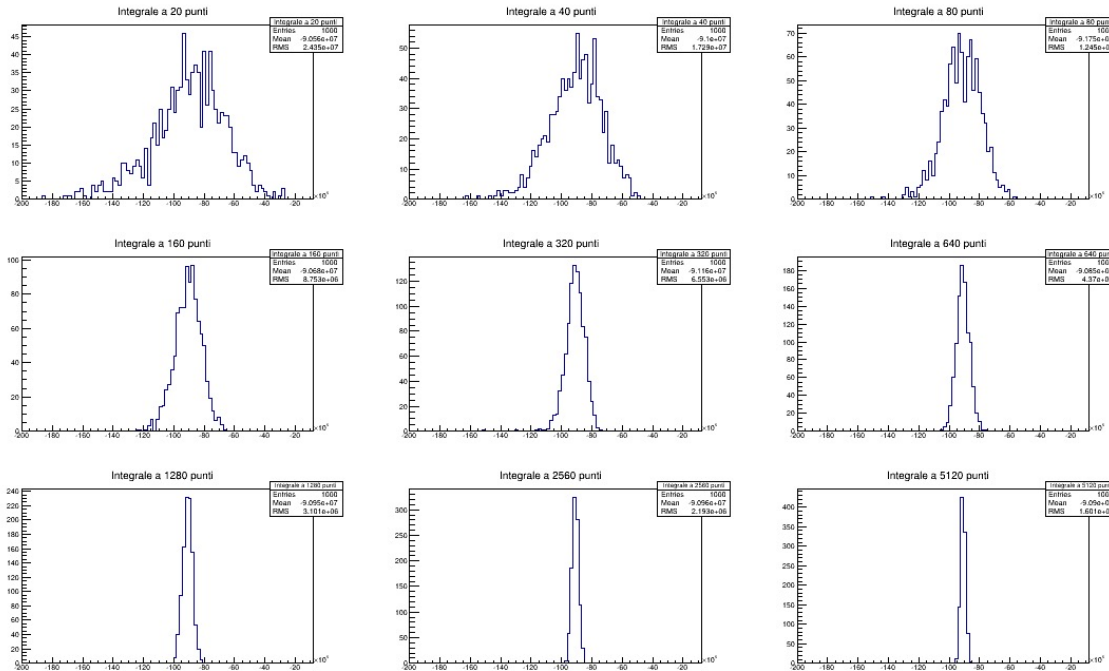
$$V(\vec{r}) = - \int_{-R}^R dx' \int_{-R}^R dy' \int_{-R}^R dz' G \frac{\rho(\vec{r}')}{|\vec{r} - \vec{r}'|} f(\vec{r}')$$

$$f(\vec{r}') = \begin{cases} 0 & \text{se } |\vec{r}'| > R \\ 1 & \text{se } |\vec{r}'| < R \end{cases}$$

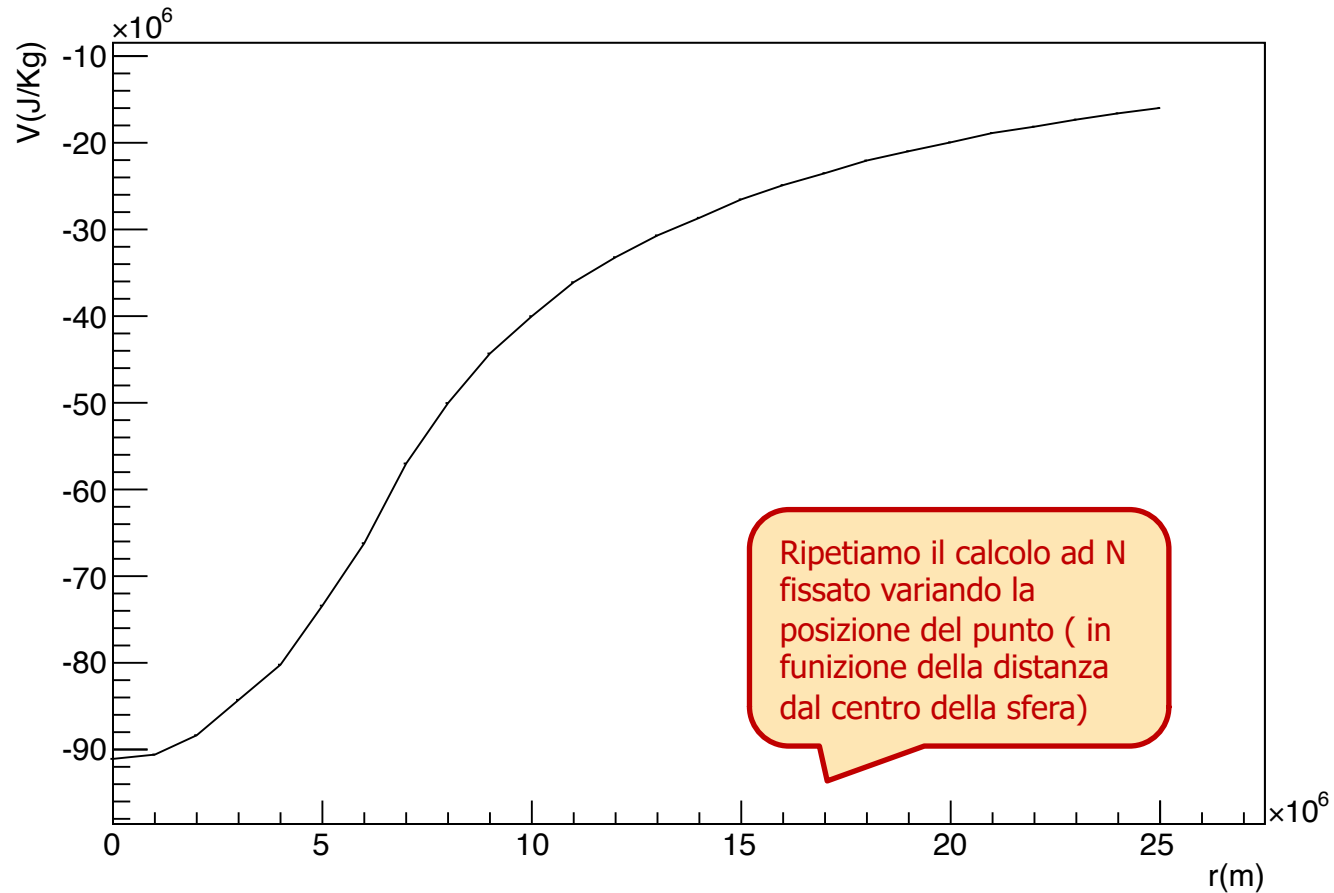
Posso utilizzare il metodo della media :

- ☐ Estraggo (uniformemente) una terna di punti in un cubo di lato $2R$ che contenga la terra
- ☐ Per ogni terna calcolo il valore della funzione (che in questo caso e' una funzione scalare)
- ☐ Eseguo i punti precedenti un numero N di volte e calcolo la media della funzione nei punti estratti
- ☐ Moltiplico questo numero per il volume di estrazione $(2R)^3$

Interessa un integrale 3D ?



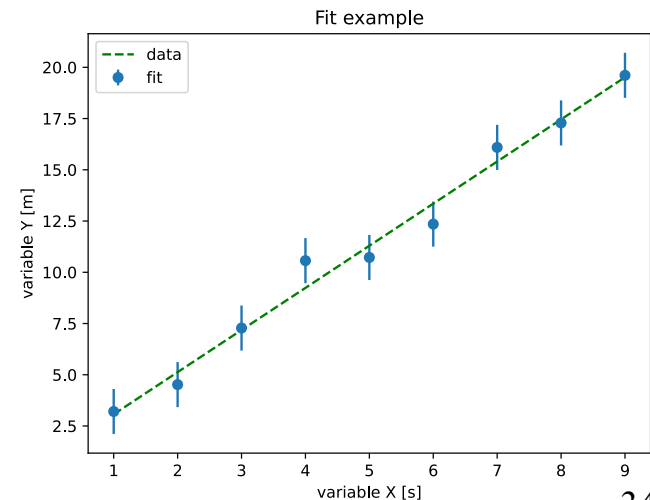
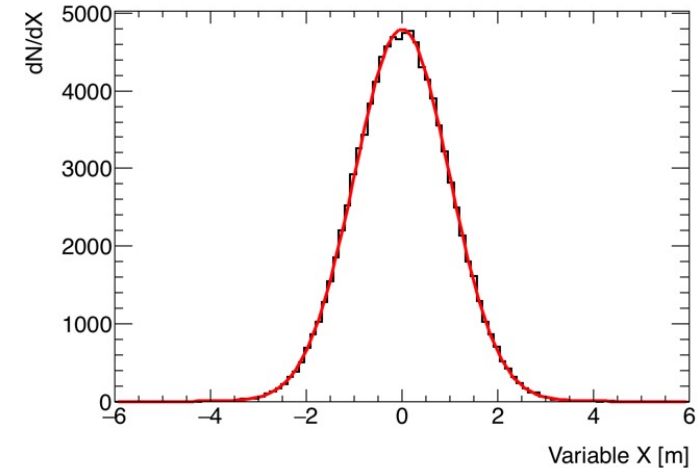
Posso anche disegnare una funzione integrale !!!



Esercizio 10.2 : eseguire un fit

L'esercizio 10.2 ci dà lo spunto per un primo approfondimento su cosa significa fare un fit. Possiamo considerare due casi :

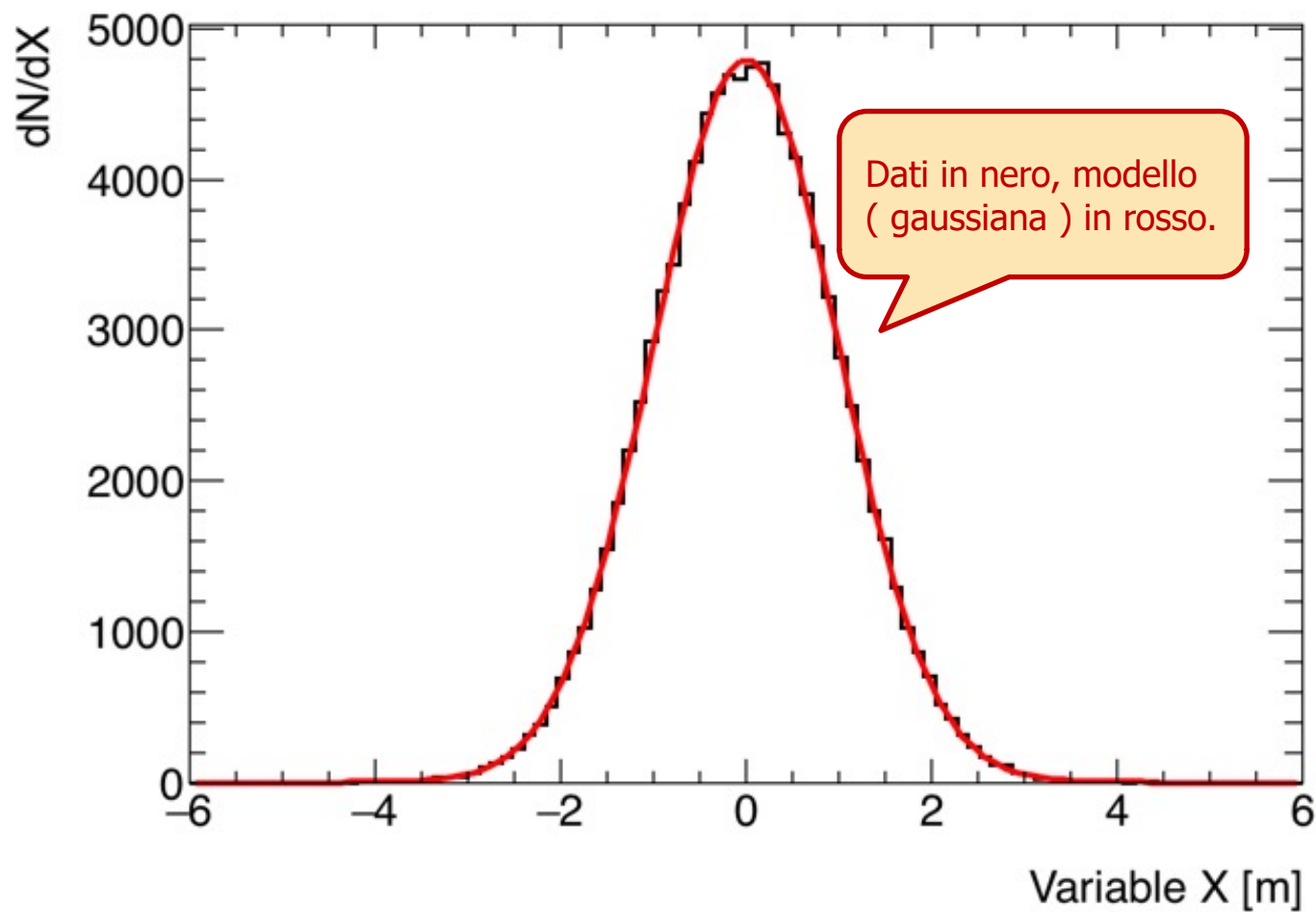
1. Abbiamo effettuato N misure di una certa quantità X e vogliamo capire se un certo modello teorico (funzione) descrive la distribuzione di X .
2. Abbiamo N coppie di misure (X, Y) e vogliamo capire se un certo modello teorico (funzione) descriva l'andamento osservato di Y al variare di X



Eseguire un fit : caso 1

- ❑ Abbiamo una serie di misurazioni di una osservabile e costruiamo una distribuzione (tipicamente un istogramma)
- ❑ Vogliamo capire se il set di dati potrebbe provenire da una data distribuzione teorica sottostante (tipicamente descritta da una funzione):
 - ❑ Primo: per una data forma funzionale assunta vogliamo provare ad estrarre i parametri del modello che danno il miglior accordo con i dati
 - ❑ Secondo: vorremmo anche verificare se il modello che stiamo assumendo è accettabile: il modello rappresenta davvero i miei dati?
- ❑ Fare un fit significa cercare l'insieme di parametri del modello che offre il miglior accordo con i dati
 - ❑ Necessità di definire uno stimatore (figura di merito per decidere cosa è meglio)
 - ❑ Necessita di un minimizzatore (algoritmo per minimizzare la figura di merito): varia in modo intelligente i valori dei parametri nello spazio (multidimensionale) dei parametri fino a trovare il miglior valore dello stimatore
- ❑ In genere, sul mercato sono disponibili pacchetti per eseguire queste operazioni in modo ottimale: ROOT (ad esempio) fornisce l'infrastruttura per eseguire questo chiamando Minuit come minimizzatore

Esercizio10.2 : eseguire un fit



Quale figura di merito minimizzare ?

- ❑ Possiamo costruire lo stimatore a piacere, ma ce n'è uno che ha delle proprietà interessanti, il χ^2

$$\chi^2 = \sum_{i=1}^n \frac{(o_i - e_i)^2}{e_i}$$

- ❑ Dove o_i è il numero osservato di eventi in un bin mentre e_i è il numero previsto (dal modello) di eventi in un bin (ed n è il numero di bin riempiti dell'istogramma).
- ❑ Perché χ^2 ? Perché conosciamo la distribuzione di questa variabile!
 - ❑ Possiamo usarlo per stimare la bontà del fit
- ❑ Lo stimatore costruito sopra è distribuito come la seguente variabile (k qui è n-# di parametri della funzione di adattamento):

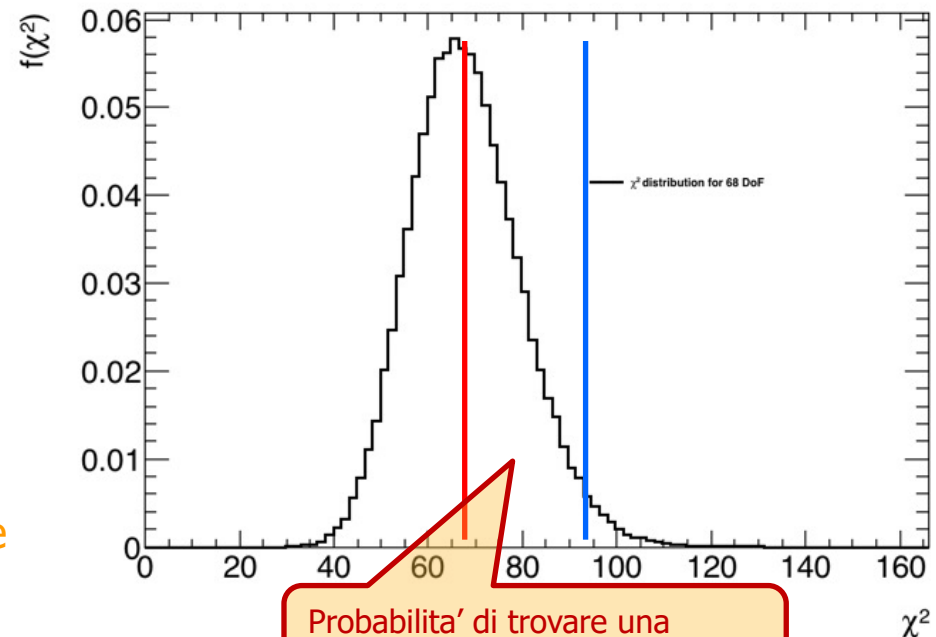
$$\chi^2(k) = \sum_{i=1}^k x_i^2 = x_1^2 + \dots + x_k^2$$

- ❑ dove x_i sono variabili indipendenti distribuite come $N(0,1)$

Il test di χ^2 (test d'ipotesi)

- ❑ Definire un'ipotesi nulla: il set di dati è una rappresentazione di una variabile distribuita come una distribuzione teorica $m(x, \mathbf{p})$ dove $\mathbf{p}$ è un insieme di parametri.
- ❑ Definire il livello di confidenza che possiamo tollerare (tipicamente 5%)
- ❑ Consideriamo la distribuzione $f(\chi^2)$ del χ^2 per il dato numero di gradi di libertà e calcoliamo la soglia χ^2_α per la quale $P(\chi^2 > \chi^2_\alpha) = 0.05$
- ❑ Calcoliamo il valore di χ^2_m e rigettiamo l'ipotesi nulla se $\chi^2_m > \chi^2_\alpha$

- ❑ Assumiamo un fit con 68 DoF: troviamo $\chi^2_m = 70$ (linea rossa) che è chiaramente molto inferiore a χ^2_α (linea blu): non posso rifiutare l'ipotesi nulla quindi il modello può essere accettato
- ❑ Se $\chi^2_m \gg \chi^2_\alpha$ è molto improbabile trovare un altro set di dati più incompatibile con il modello, quindi posso rifiutare l'ipotesi nulla
- ❑ Operativamente utilizzare la funzione Prob() per calcolare $P(\chi^2 > \chi^2_m)$ e confrontarlo con 0.05: se $P(\chi^2 > \chi^2_m) > 0.05$ non posso rifiutare il modello quindi va bene



Probabilità di trovare una misura più incompatibile (prob)

Esempi di fit

```
// put data into an histogram

TH1F data_small ("data_small","data_small",100, -6, 6 ) ;
for ( int k = 1 ; k < 100 ; k ++ ){ data_small.Fill( ... ); };

// create the fitting function ( already in ROOT )

TF1 * gaus_small = new TF1("gaus","gaus",-10,10);

// setup plots

TCanvas * can_small = new TCanvas("test_small","test_small");
can_small->cd();

// set cosmetics

data_small.Draw();

// Performt the fit and access the results

TFitResultPtr res_small = data_small.Fit(gaus_small,"S","", -10,10);
res_small->Print();
cout << "Fit small data : Prob = " << gaus_small->GetProb() << endl;
```

Esempi di fit

FCN=19.9441 FROM MIGRAD STATUS=CONVERGED 76 CALLS 77 TOTAL
EDM=4.0448e-07 STRATEGY= 1 ERROR MATRIX ACCURATE

EXT NO.	PARAMETER NAME	VALUE	ERROR	STEP SIZE	FIRST DERIVATIVE
1	Constant	3.79521e+00	6.17374e-01	9.53237e-04	-1.07965e-03
2	Mean	1.71244e-01	1.58176e-01	3.51747e-04	-1.27424e-03
3	Sigma	1.13353e+00	2.03254e-01	9.86157e-05	4.06094e-03

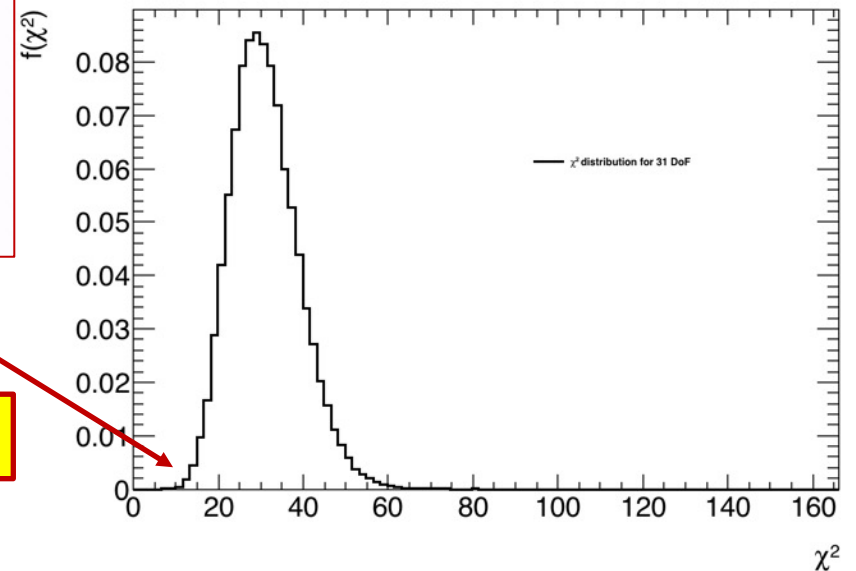
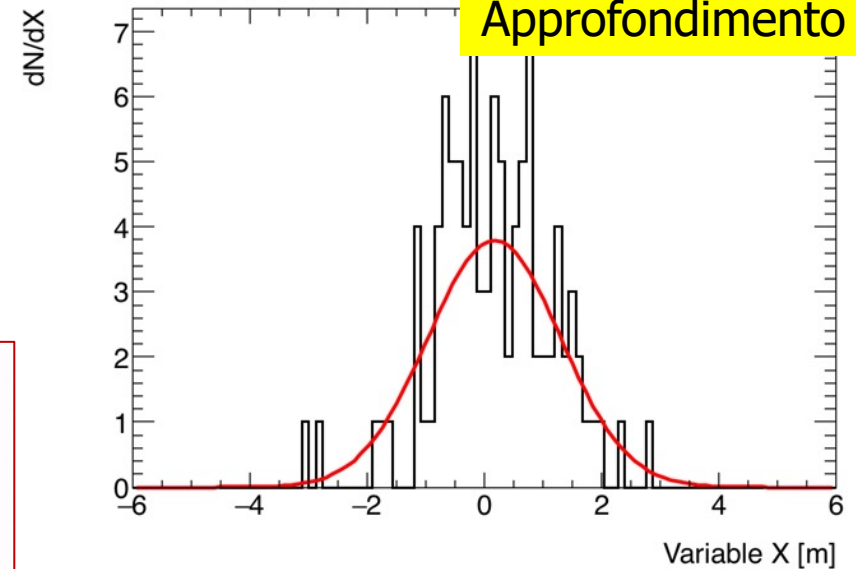
Minimizer is Minuit / Migrad

Chi2	=	19.9441		
NDf	=	31		
Edm	=	4.0448e-07		
NCalls	=	77		
Constant	=	3.79521	+/- 0.617374	
Mean	=	0.171244	+/- 0.158176	
Sigma	=	1.13353	+/- 0.203254	(limited)

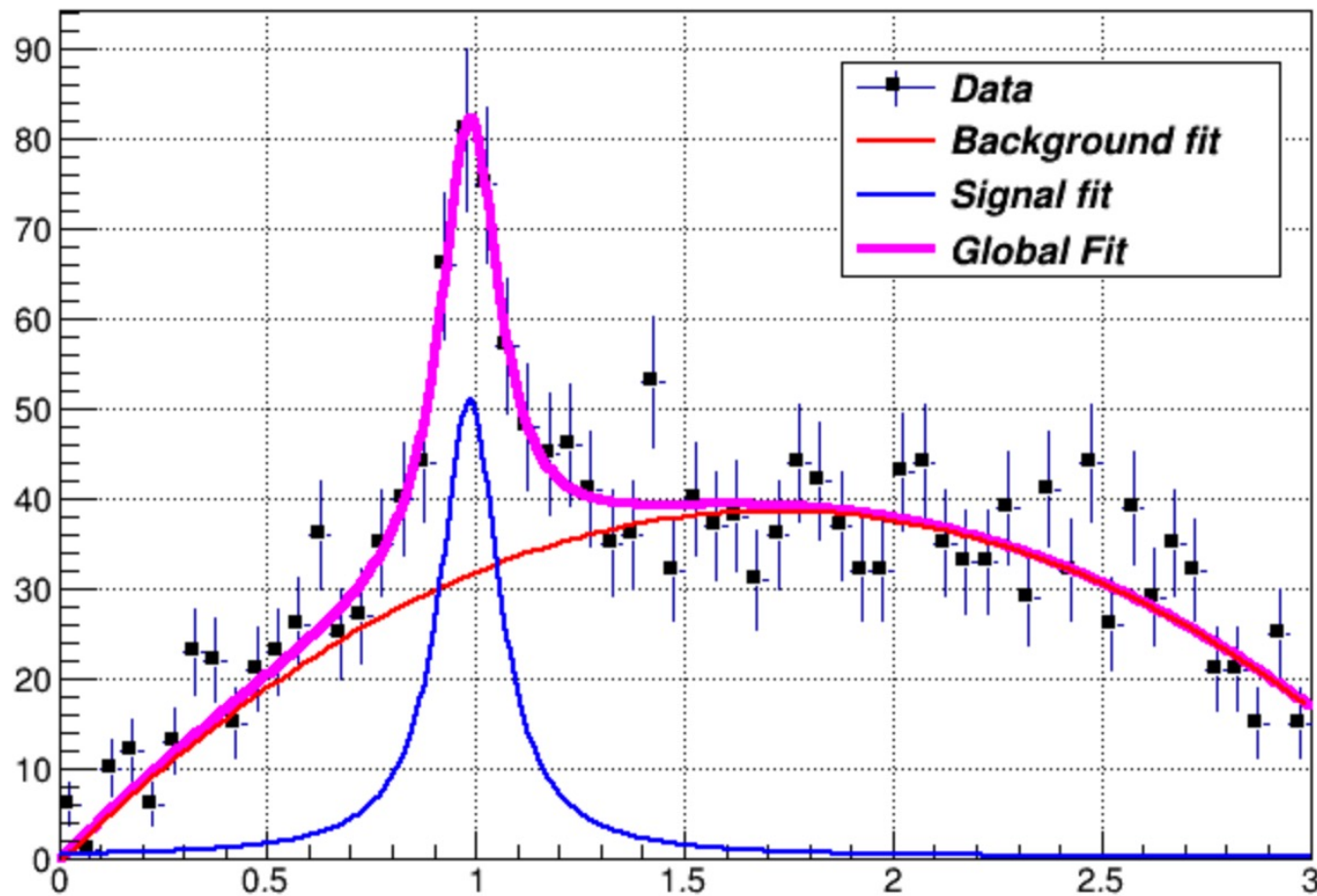
Fit small data : Prob = 0.936994

Good fit !

Approfondimento 1



Lorentzian Peak on Quadratic Background



Un fit un leggermente piu' complicato

```
// Quadratic background function

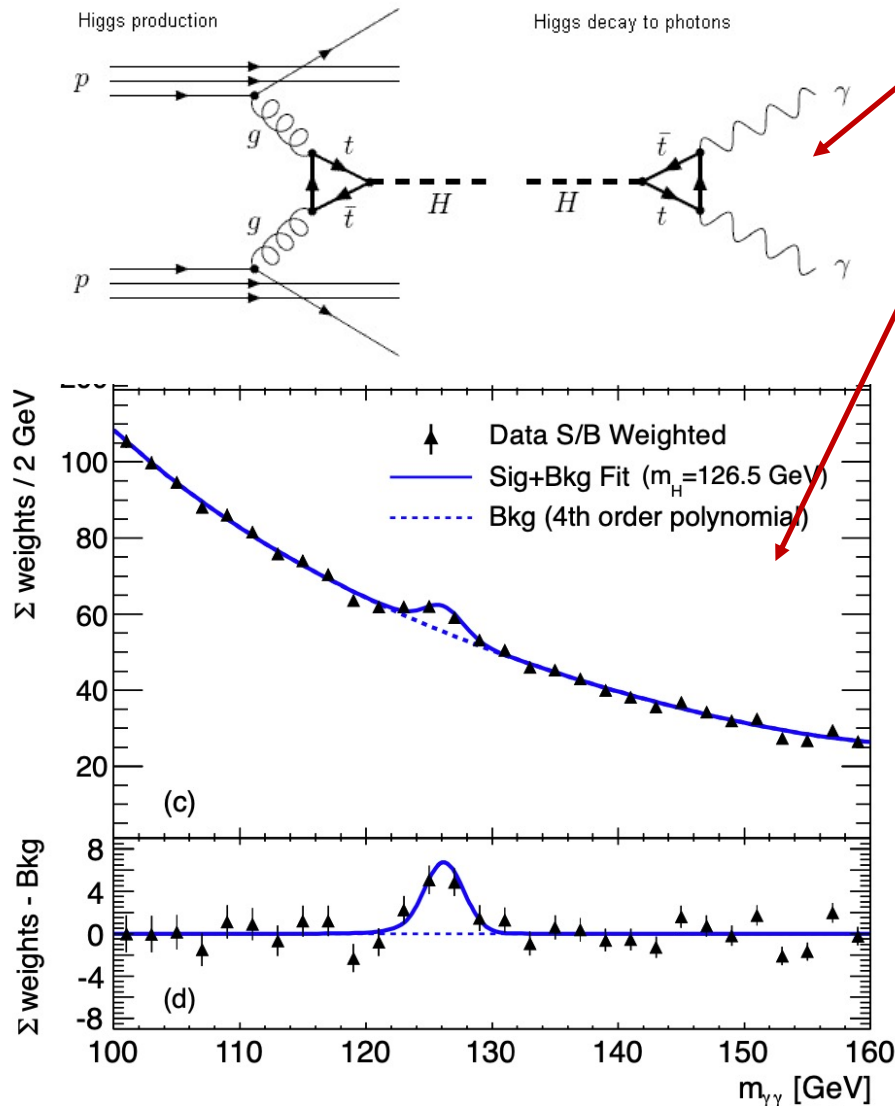
Double_t background(Double_t *x, Double_t *par) {
return par[0] + par[1]*x[0] + par[2]*x[0]*x[0];
}

// Lorentzian Peak function
Double_t lorentzianPeak(Double_t *x, Double_t *par) {
return (0.5*par[0]*par[1]/TMath::Pi()) / TMath::Max(1.e-10, (x[0]-
par[2])*(x[0]-par[2])+ .25*par[1]*par[1]);
}

// Sum of background and peak function
Double_t fitFunction(Double_t *x, Double_t *par)
{ return background(x,par) + lorentzianPeak(x,&par[3]); }

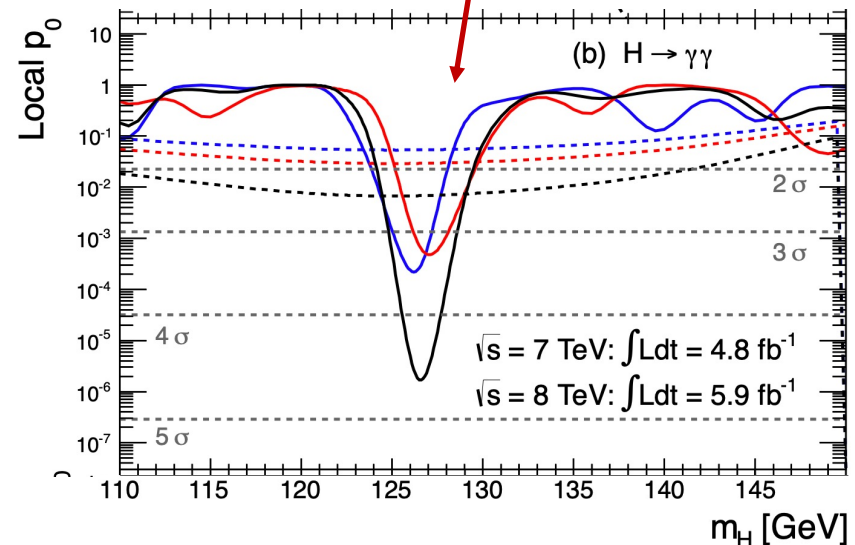
void FittingDemo() {
TH1F *histo = new TH1F("example_9_1", "Lorentzian Peak on Quadratic
Background",60,0,3);
// create a TF1 with the range from 0 to 3 and 6 parameters
TF1 *fitFcn = new TF1("fitFcn",fitFunction,0,3,6);
histo->Fit("fitFcn");
```

Un fit davvero interessante : misurare la massa dell'Higgs !



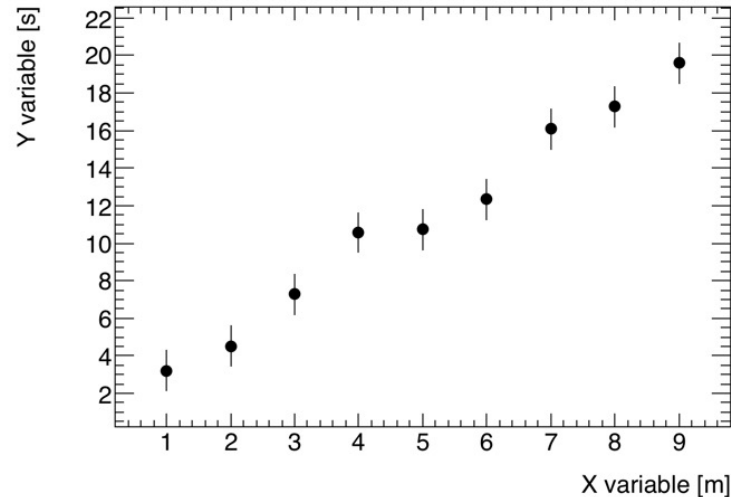
La scoperta di Higgs è stata un test di ipotesi:

- ❑ Selezioniamo eventi con due fotoni nello stato finale
- ❑ Scegliamo una variabile discriminante (la massa invariante $m_{\gamma\gamma}$ dei due fotoni)
- ❑ Costruiamo un modello per la forma $m_{\gamma\gamma}$ del fondo (funzione esponenziale)
- ❑ Ipotesi nulla: ciò che vedo nei dati è compatibile con l'ipotesi del solo fondo
- ❑ Scegliamo una test statistico e calcoliamo la probabilità di vedere qualcosa di più incompatibile con l'ipotesi nulla rispetto alla misurazione



Eseguire un fit : caso 2

- ❑ In laboratorio si misura una variabile Y e si disegnano le misurazioni in funzione di un'altra variabile X



- ❑ Vorrei verificare se il mio modello (una funzione) concorda con le misurazioni
 - ❑ Come prima, il modello è parametrico (cioè conosco la forma funzionale ma non i valori effettivi dei parametri nel modello che saranno determinati dall'adattamento ai dati)

$$\chi^2 \equiv \sum_{i=1}^n \frac{(x_i - x_{ti})^2}{\sigma_i^2}$$

- ❑ Come sopra, questo stimatore di distribuisce come χ^2 con Npunti-Npara DoF!

A slightly different case

```
{  
  
    ifstream file ;  
    file.open("data.dat") ;  
  
    TGraphErrors * mygr = new TGraphErrors();  
  
    double x,y,err;  
  
    for ( int k =0 ; k < 9 ; k++ ) {  
        file >> x >> y >> err ;  
        mygr->SetPoint(k, x, y );  
        mygr->SetPointError(k,0, err);  
    }  
  
    file.close();  
  
    // Draw  
  
    TCanvas * can_tgraph = new TCanvas("can_tgraph","can_tgraph");  
    can_tgraph->cd();  
    mygr->GetXaxis()->SetTitle("X variable [m]");  
    mygr->GetYaxis()->SetTitle("Y variable [s]");  
    mygr->Draw("AP");  
  
    // define the fitting function, set initial values and fit. Then see results  
  
    TF1 * myfit = new TF1("myfit","[0]*x + [1]",0,10);  
    cout << "First parameter " << myfit->GetParameter(0) << endl;  
    myfit->SetParameter(0,1.5);  
    TFitResultPtr res1 = mygr->Fit(myfit,"S","",1,9);  
    cout << "First parameter " << myfit->GetParameter(0) << endl;  
  
    res1->Print();  
    cout << myfit->GetProb() << endl;  
}
```

Execute with :
root test_root.C

Ipotesi nulla : correlazione lineare

A slightly different case

Approfondimento 1

```
FCN=3.30513 FROM MIGRAD   STATUS=CONVERGED   31 CALLS   32 TOTAL
                        EDM=1.36386e-22   STRATEGY= 1   ERROR MATRIX ACCURATE

EXT PARAMETER
NO.   NAME      VALUE      ERROR      STEP      FIRST
  1   p0       2.05529e+00  1.42009e-01  6.60136e-05  2.35453e-10
  2   p1       1.01647e+00  7.99131e-01  3.71479e-04  4.48298e-11
First parameter 2.05529
```

Minimizer is Minuit / Migrad

Chi2 = 3.30513

NDf = 7

Edm = 1.36386e-22

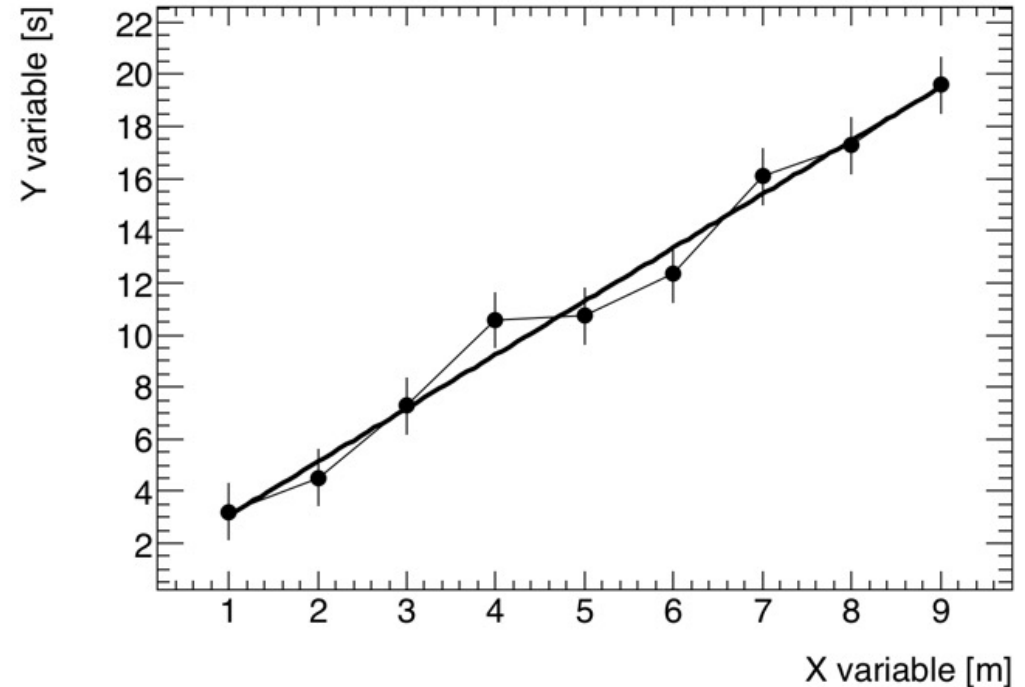
NCalls = 32

p0 = 2.05529 +/- 0.142009

p1 = 1.01647 +/- 0.799131

0.855414

La Prob è grande, non si può rigettare
l'ipotesi nulla, accettiamo il modello



Attenzione : la minimizzazione e' un'arte !

- ❑ Normalmente non si può semplicemente eseguire un fit alla cieca e fidarsi dei risultati.
- ❑ Spesso potremmo essere costretti a fornire all'algoritmo di minimizzazione un insieme di valori iniziali "ragionevoli" per i parametri:
 - ❑ Questo non significa che stiamo barando, in questo caso l'algoritmo convergerà più velocemente al minimo

```
cout << "First parameter " << myfit->GetParameter(0) << endl;  
myfit->SetParameter(0,1.5)
```

Be flexible : like gnuplot ?

Approfondimento 1

```
#definisce il terminale
set term x11

# definizione dei titoli per gli assi e per il grafico
set title "titolo del grafico"
set xlabel "X variable [m]"
set ylabel "Y variable [s]"

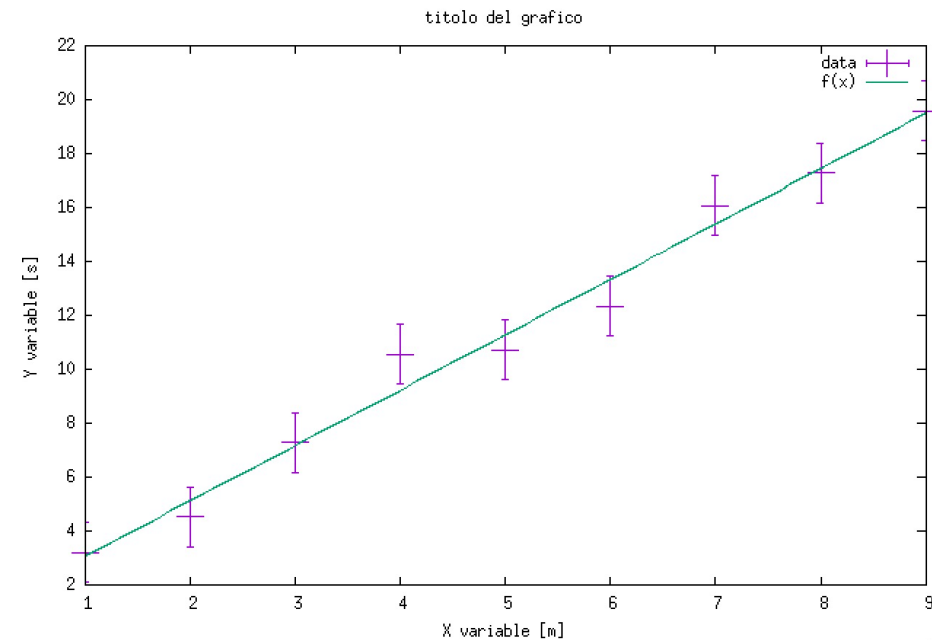
# definizione della funzione
f(x) = a + b * x

# valori delle costanti
a=1.5; b=4.5;

fit f(x) 'data.dat' u 1:2:3 yerr via a,b

pl 'data.dat' u 1:2:3 title 'data' w ye points 3, f(x)
```

Execute with :
gnuplot
gnuplot>load "test_gnuplot.gnu"



Be flexible : like python ?

```
import numpy as np
from numpy import pi, r_, loadtxt
import matplotlib.pyplot as plt
from scipy.optimize import curve_fit

data = loadtxt('data.dat')
x = data[:, 0]
y = data[:, 1]
err = data[:, 2]

def func(x, a, b ):
    return a + b * x

plt.errorbar(x, y, yerr=err , label='data' , fmt='o')

popt, pcov = curve_fit(func, x, y, sigma=err)
print "Best fit parameters" , popt

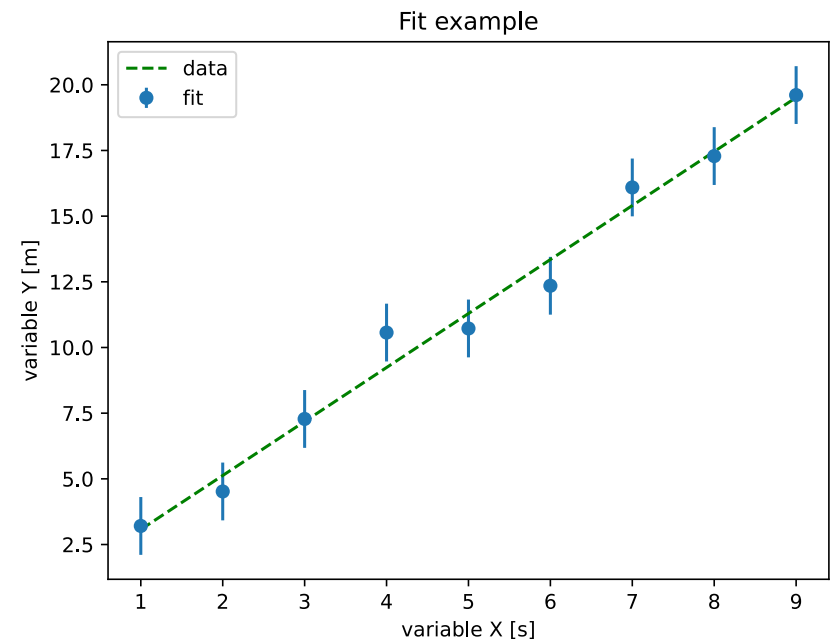
chisq = sum(((y-func(x,*popt))/err)**2)
print "Chisq = " , chisq

plt.plot(x, func( x , *popt), 'g--', label='fit')

plt.title("Fit example")
plt.xlabel("variable X [s]")
plt.ylabel("variable Y [m]")
plt.legend(('data', 'fit'))

plt.show()
```

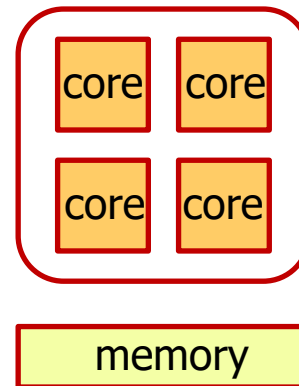
Execute with :
python test_python.py



- ❑ In linea di principio potremmo continuare con molte altre opzioni (guardare pyROOT per esempio o matlab)
- ❑ Il linguaggio perfetto o lo strumento perfetto per tutto non esistono, diffidate dei fondamentalisti
- ❑ Siate flessibili, intelligenti ed eclettico: scegliamo un linguaggio, progettiamo il codice e utilizziamo gli strumenti che meglio si adattano alle nostre esigenze.
 - ❑ Tipicamente un mix di linguaggi (eg C++ e Python)

Esercizio10.2 : velocizzare la generazione

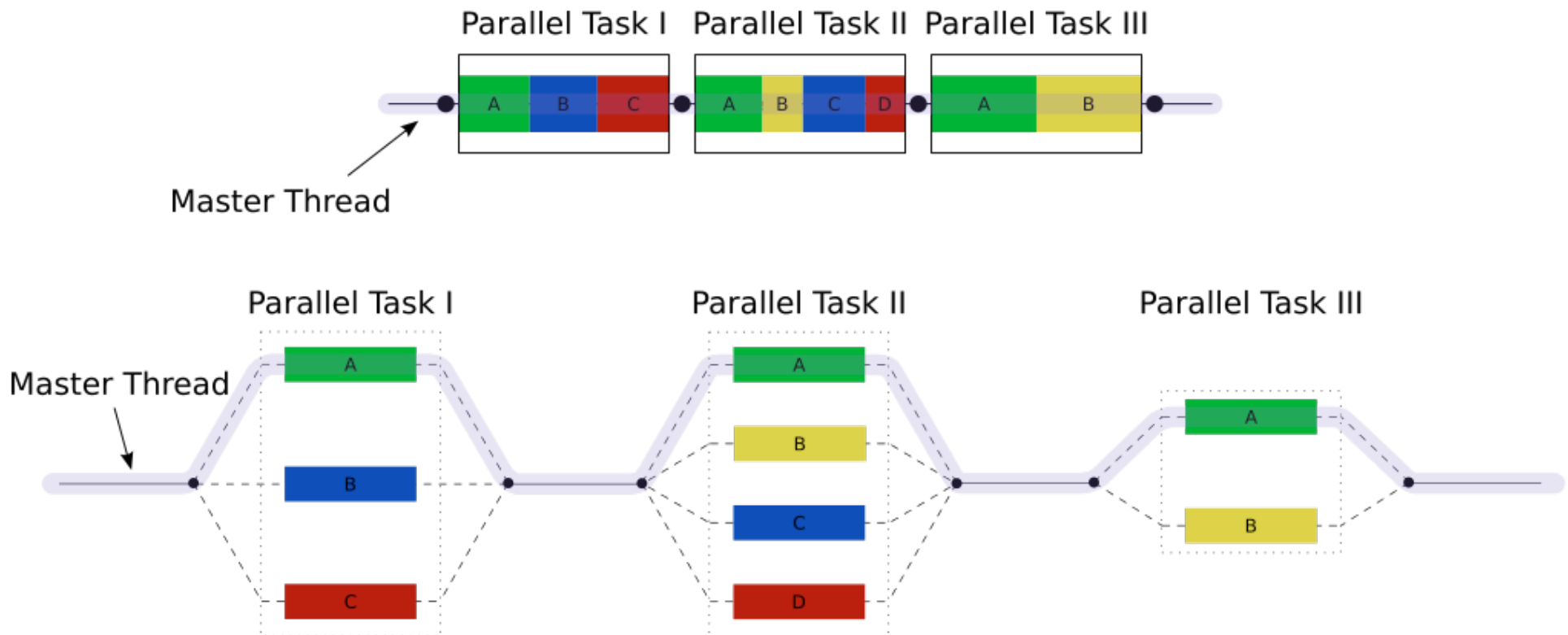
- ❑ I moderni processori sono costituiti da piu' computing units (cores) in ogni CPU : i nostri programmi sono tipicamente eseguiti da un solo core mentre normalmente gli altri stanno a guardare.
 - ❑ E' possibile scrivere codice in modo che il carico sia distribuito su tutti i cores di una macchina ?



- ❑ Si puo' utilizzare per esempio openMP che permette di generare diversi threads che vengono eseguiti in parallelo su cores diversi

The fork/join model

- ❑ OpenMP programs start with a single thread; the master thread
- ❑ At start of parallel region master creates team of parallel “worker” threads (FORK)
- ❑ Statements in parallel block are executed in parallel by every thread
- ❑ At end of parallel region, all threads synchronize, and join master thread (JOIN)



A simple parallelization example : openMP

OpenMC comes as a standard API (*) for writing shared memory parallel applications in C, C++, and Fortran

OpenMP API consists of:

- ❑ Compiler Directives

- ❑ Ex `#pragma omp parallel` : to define the parallel part of the code

- ❑ Runtime subroutines/functions

- ❑ `omp_get_num_procs()` (returns the number of processors in the machine)

- ❑ `omp_get_thread_num()` (returns the thread number)

- ❑ Environment variables

- ❑ Export `OMP_NUM_THREADS=4`

(*) In computer programming, an application programming interface (API) is a set of subroutine definitions, protocols, and tools for building application software.

Threads

- ❑ Thread is independent sequence of execution of program code
 - ❑ Block of code with one entry and one exit
- ❑ For our purposes thread is a more abstract concept
- ❑ Threads are unrelated to Cores/CPUs
- ❑ OpenMP threads are mapped onto physical cores
- ❑ Possible to map more than 1 thread on a core
- ❑ In practice best to have one-to-one mapping.

See a couple of online examples on replit <https://replit.com/join/vefatiqcan-lcarmina>

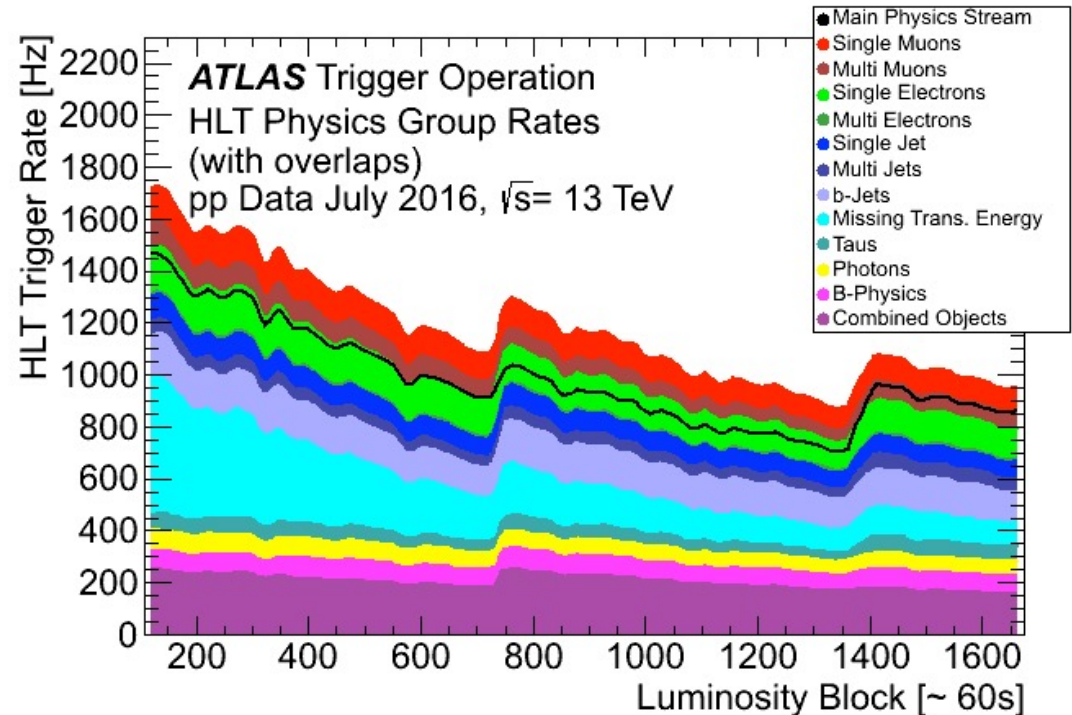
What are we talking about

❑ Complex models : a single big computation !

- ❑ Weather forecast : split atmosphere in 10^8 regions
- ❑ Need to evolve the prediction in each region say at 2 days
- ❑ Need to have a result before of course!

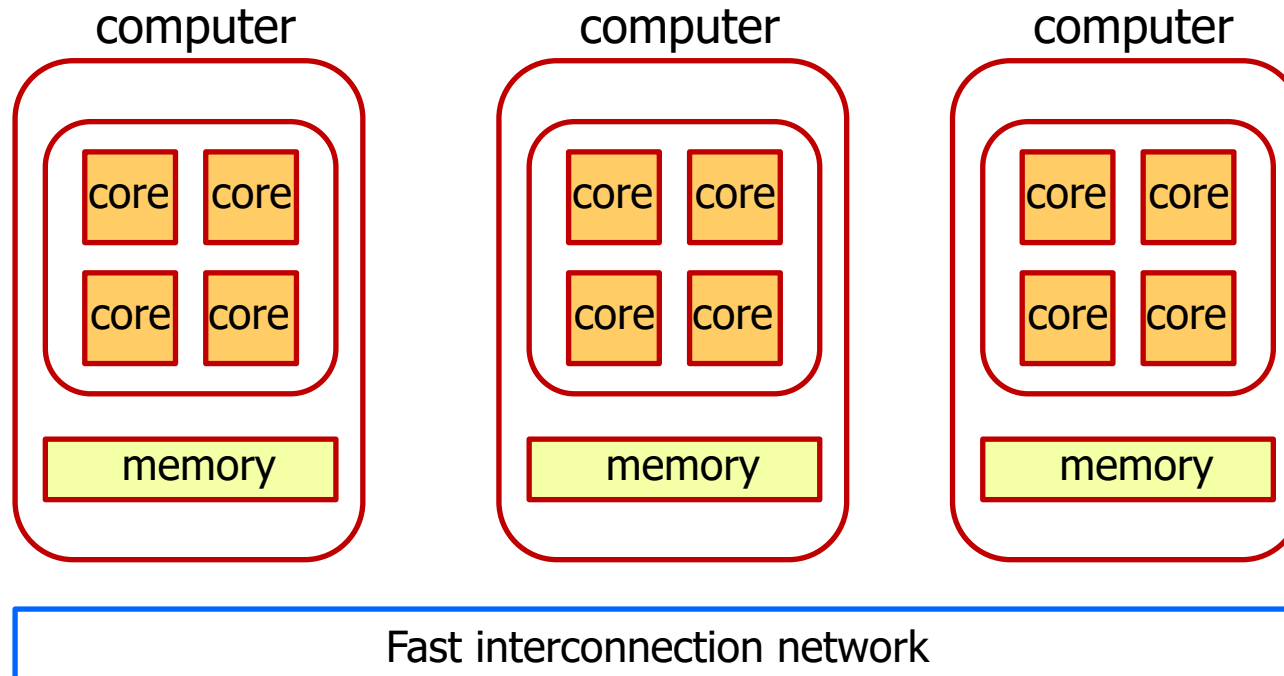
❑ Big datasets : same operations on different inputs

- ❑ Detector output rate 1000 Hz
- ❑ $1000 \text{ Hz} * 10^7 \text{ s} = 10 \text{ billion events/year}$ (1./3 data taking efficiency included)
- ❑ Say we need a processing time of 10 sec per event
- ❑ How can we keep up with the event rate ? Split into 10000 CPUs (embarrassingly parallel)



Modelli complessi : generazione parallela

Caso in cui i vari processi di soluzione di un problema complesso comunicano tra loro



Generazione parallela e batch system

Caso di parallelismo imbarazzante : le parti non comunicano tra loro (caso tipico l'analisi dei dati di un esperimento) si puo' usare un batch system (eg condor)

